

Supplementary Material for “Faster than Light: Silent Errors in the Latency Benchmarks Used to Choose Message Brokers”

Gustavo Pedro Ricou and Romaric Duvignau

This document is the supplementary material for the paper named above, and it follows that paper’s order and its names. Table S1 lists, for each section of the paper, the sections here that hold its evidence. Table S2 defines the terms both documents use, each with one meaning. The complete record of how the results were obtained is a separate document in the archived artifact, the postmortem. It retells the work in the order it happened, with every result that did not survive and the full log of the registered law campaign. Pointers such as “postmortem, S16.10” refer to its sections.

S1. THE SETUP IN FULL

The paper’s results depend on the football feed only through message size and rate, and none depends on synchronizing two clocks unless it says so.

S1.1. Testbeds and clocks

Each of three testbeds carries a different part of the evidence (Table S3). On the cloud testbed the producer and the consumer run as separate processes on one host with 8 online CPUs, and read one wall clock, `CLOCK_REALTIME`, through `time.time_ns()`. No scheduler tunable is written anywhere in the archived campaign, which is strong evidence, short of proof, that the kernel’s defaults applied, among them the derived base slice of 3 ms (Section IV-C). The clocksource was not recorded, but a measured read cost narrows down which one it was (S3). Across hosts, `chrony` reports an inter-host offset of 0.067 ms but bounds its own error at 4–7 ms per host over the 4 hosts captured, far more than the offset, so we claim no cross-host bound.

G. P. Ricou is with the School of Computer Science and Statistics, Trinity College Dublin, Dublin 2, Ireland. E-mail: pedrorig@tcd.ie.

R. Duvignau is with the Department of Computer Science and Engineering, Chalmers University of Technology, Gothenburg, Sweden. E-mail: duvignau@chalmers.se.

TABLE S1

EACH SECTION OF THE PAPER HAS ITS EVIDENCE IN NAMED SECTIONS HERE. PAPER SECTIONS BY NUMBER, THE SECTIONS OF THIS DOCUMENT THAT HOLD THEIR EVIDENCE, AND WHAT THOSE SECTIONS ADD.

Paper section	Here	What it adds
I. Introduction	S2, S3, S4	The sign check’s rejections, and both failures in full.
II. How a benchmark times a message	S1, S2	Testbeds, workload, settings, campaigns and where each timestamp is taken; the threshold sweep and what the check selects.
III. The finding	S2, S8, S6	The rejection counts; the first result the check rejected; the negatives in Kafka’s Java client and on a second processor family.
IV. The mechanism	S3, S5, S6	Every manipulation, trace and rival explanation; origin timestamps written late in other runtimes; the registered law campaign.
V. The industry’s remedy	S4, S5	The source lines, the grid tests, the registered payload manipulation, the sign channel and our own harness; the source line behind each audited tool; where the eleven tools run are recorded; the registered audit of published reports; the millisecond record timestamp on Kafka’s wire.
VI. Practical implications	S3, S7	The exposure and the repair; the broker comparison over the runs the check keeps.
VII. Related work	S9	The literature in full.
VIII. Threats and limitations	S3, S4, S6, S9	The rival explanations and the clocksource bound; the benchmark’s distributed mode; the law campaign’s verdicts; why a skew threshold does not transfer to a loaded machine.
IX. Conclusion	none	Nothing; the conclusion draws on the sections above.

TABLE S2

EACH TERM HAS ONE MEANING, HERE AND IN THE PAPER. THE TIMESTAMPS ARE THOSE OF EQUATIONS 1 AND 2; BOTH PROCESSES READ ONE CLOCK UNLESS A RESULT SAYS OTHERWISE.

Term	Meaning
Failure 1	A latency timed from the acknowledgment turns negative when the thread recording the acknowledgment is scheduled late by more than the end-to-end latency (Sections III and IV).
Failure 2	A millisecond timestamp and a positivity filter delete, without counting them, the sub-millisecond samples that cross no millisecond boundary (Section V).
condition	One point of the experimental design: a broker, a publish rate, a message size, a background load and a consumer pattern.
cell	A condition together with a workload; where the workload is fixed, the two coincide. A run executes a cell once, and a replicate is a further run of it.
configuration	One setting of a manipulation that compares settings differing in one variable.
utilization ρ	The fraction of processor time busy on the host, measured over the run.
core count k	The number of cores the background load occupies.
load geometry	How a utilization is reached: the load concentrated on few cores or spread over many.
occupancy p	The probability that a timestamping thread is preempted and waiting for a core.
scheduling delay	The time a thread that is ready to run spends queued before a core runs it.
end-to-end latency D	$t_{\text{recv}} - t_{\text{pub}}$: from the publish call to the consumer holding the record.
publish latency A	$t_{\text{ack}} - t_{\text{pub}}$: from the publish call to the producer observing the broker's acknowledgment.
transport proxy S	$t_{\text{recv}} - t_{\text{ack}} = D - A$: a message timed from the acknowledgment. It is not a causal chain.
event-time latency E	$t_{\text{out}} - t_{\text{sched}}$: from the event falling due to the consumer having handled it, the sum of the publish delay, A , S and the processing-time latency.
processing-time latency	$t_{\text{out}} - t_{\text{recv}}$: the consumer's own work on the record after t_{recv} .
publish delay	$t_{\text{pub}} - t_{\text{sched}}$: how late the publish call ran after the event was due.
true end-to-end latency T_{true}	The interval D would report if its timestamps were arbitrarily fine. A difference of two timestamps below zero is a negative span.
timestamp resolution τ	The smallest difference a timestamp can express: one millisecond in the OpenMessaging Benchmark.
grid	The instants, τ apart, at which a timestamp changes value.
q -lattice	The retentions $0, 1/q, \dots, 1$ a paced producer can reach when its publish interval over τ is a/q in lowest terms (S4.2).
retention	The fraction of the samples a tool takes that survive into what it reports.
positivity filter	A rule that admits a sample to a histogram only when its latency is strictly positive, as at <code>WorkerStats.java:95</code> in the OpenMessaging Benchmark.
registered	Committed in public before the data that test it.
sign check	A run is rejected if more than 1% of its events carry a negative value in any latency component, or if the median of any component is negative. A condition is usable only if all of its runs survive (Section II-C).

TABLE S3

EACH TESTBED CARRIES A DIFFERENT PART OF THE EVIDENCE. MACHINES, SOFTWARE, AND WHAT RESTS ON EACH TESTBED.

Testbed	Machines	Kernel and software	What rests on it
Cloud	Four Oracle Cloud VM.Standard.E5.Flex instances on a private network: three brokers and one driver.	Ubuntu 22.04, kernel 6.8.0-1057-oracle (EEVDF scheduler), a 1 ms tick; CPython 3.10; chrony-disciplined clocks, logged every minute.	The findings of Sections III to V, except the law campaign's.
Workstation	One Windows machine running the brokers in Docker Desktop containers, inside a WSL2 virtual machine on Microsoft's hypervisor.		The sign check only; no attribution rests on it (S2).
Law campaign	Three Microsoft Azure pairs in one region, each a driver with eight virtual CPUs and a broker with two: two x86 pairs (AMD EPYC 9V74) and one Arm pair (Neoverse N2).	The provider's stock kernels, 6.8.0-1064 and 6.8.0-1065; for the tick block, three kernels built from one source tree that differ in the tick alone, 1, 4 and 10 ms.	The registered law campaign (S6).

S1.2. Workload

Three properties of the football feed drive the experimental choices: it is sparse, it is bursty with one burst at a known instant, and its concurrency is bounded. The feed is StatsBomb's open event data [S1]. Our characterization covers 3,315 of its matches, and the campaigns replay eleven of them in order at a chosen publish rate. A match produces a median of 3,507 events over about 8,412 s, a mean of 0.415 events per second. A match's busiest ten-second window holds 2.70 events per second at the median. Each of the eleven replayed plans opens with at least five events at one instant: two lineup records,

TABLE S4

THREE OF THESE SETTINGS WERE LEARNED FROM THE DEFECTS THEY CAUSED. EVERY CLIENT SETTING THAT CAN MOVE A LATENCY NUMBER, AND WHY IT HOLDS ITS VALUE. LEARNED: FOUND THROUGH A DEFECT IT CAUSED, NOT CHOSEN IN ADVANCE.

Setting	Value	Why
<i>Kafka producer</i>		
acks	all	Durability matched to Redis’s synchronous XADD. A weaker acknowledgment would compare different guarantees.
linger.ms	0	No artificial batching delay: on a sparse workload any linger would dominate the quantity measured.
max-inflight	64	Learned. At our producer’s default of 1 the client blocks on a round trip per event, which makes the comparison one of harnesses (S8).
compression	none	Football events are small JSON. Compression would add CPU work to the timed path for no representative benefit.
ack-stamp	callback	The default, deliberately asymmetric with Redis. The inline mode measures what the asymmetry costs (S7).
<i>Redis producer</i>		
publish workers	pool	Non-blocking dispatch keeps the publish loop on schedule; publish and acknowledgment are timestamped in the worker.
<i>Consumers</i>		
Redis ack-batch	200	Learned. Acknowledging each message adds a round trip per message: at 20 ms of delay, median event-time latency 4,138 ms against 103 ms batched (S7).
Redis count	200	Read batch, matched to the acknowledgment batch so neither dominates.
Kafka poll-timeout	1000 ms	The client already amortizes fetches. No per-record round trip exists to remove.
<i>Replay</i>		
speed-up	derived	Learned. Plans carry a built-in 120× compression, so the flag is not the rate. The rate is derived from the plan and checked against wall time before every campaign (S8).
window	60–600 s	Swept on purpose, because the number of events a run contains is itself a variable (S8).

two half-start markers and the kick-off pass. Across all 3,315 matches, 575 open with four. That burst is the first thing the producer publishes, before its client has warmed up, which turns a one-off client start-up cost into the median of a short run (S8).

Kick-off times are recorded, so concurrency is observed: across 2,346 kick-off instants at most 12 matches kick off together, and at most 21 are in play at once. The historical broker comparison, E1, ran at $N \in \{1, 9, 10, 12\}$ feeds. The first is the median number of matches per kick-off instant, and also the upper quartile. The middle two are the final matchdays of an 18-team and a 20-team league, and the last is the observed maximum. The powered campaigns, sized in advance for statistical power, kept 1, 9 and 12 (S7). The law campaign replays a steady 50 messages a second instead, except its block M0, which replays a match’s bursts.

S1.3. Every setting that can move a latency number

Two clients are comparable only when matched on what they do, because Kafka and Redis expose different settings and matching them by name builds in an asymmetry. Published Kafka comparisons are frequently not comparable for this reason. Client configuration and acknowledgment semantics differ silently between the systems compared [S2]. Kafka’s producer buffers and pipelines by default, while our Redis producer dispatches to a worker pool. Allowed only one request outstanding, the Kafka client would wait a broker round trip per event and the Redis client would not, a difference in the harness that a reader would attribute to the broker. Table S4 states every client setting that can move a latency number, and why it holds its value.

S1.4. Which campaign settles which claim

Each campaign manipulates one variable and holds the others fixed, so each settles one claim and no claim rests on a single cell. Table S5 maps each claim to the campaign behind it, the runs per cell and the section here that reports it. H1, H2 and H3 label hypotheses the campaigns tested, and E1 is the historical broker comparison on the cloud testbed. Codes beginning E-A name the campaigns behind the mechanism of S3, and E-C3 and E-C4 those behind acknowledgment timestamping. A code ending in b names a second campaign of the same manipulation. The injected delay cancels in the transport proxy, whose two timestamps it shifts equally, so the delay sweep is a null result only (S8).

TABLE S5

NO CLAIM RESTS ON A SINGLE CELL. EACH CLAIM, THE CAMPAIGN BEHIND IT, THE RUNS PER CELL, AND THE SECTION HERE THAT REPORTS IT. WHERE THE SIGN CHECK REMOVES RUNS, S2 BOUNDS THE SELECTION THIS INTRODUCES.

Claim	Campaign	Runs per cell	Here
Sign-check rejections	both testbeds	2,266 runs in total	S2
Rate against utilization (H2)	E-A, E-A3	25 per load level	S3
Mixture structure	E-A3, nine levels	25 per level	S3
Priority at fixed utilization	E-A5, E-A5b, E-A7	25 per configuration	S3
Load geometry	E-A6, both geometries	25 per configuration	S3
Traced run-queue stalls	E-A9	25 per configuration	S3
T_{true} sweep	E-A10, four payloads	25 per level	S3
Acknowledgment timestamping (H3)	E-C3, replicated in E-C4	10, then 30 per cell	S3, S7
Retention under the positivity filter	the benchmark, instrumented	223 runs in total	S4
Broker comparison	powered, replicated	15, then 8 per cell	S7
Injected delay (H1)	co-located against network	15 per delay level	S8
Per-run start-up cost	window sweep, twice	3 per window	S8
Historical comparison	E1	8–36, 52.8–100% kept	S7, S8

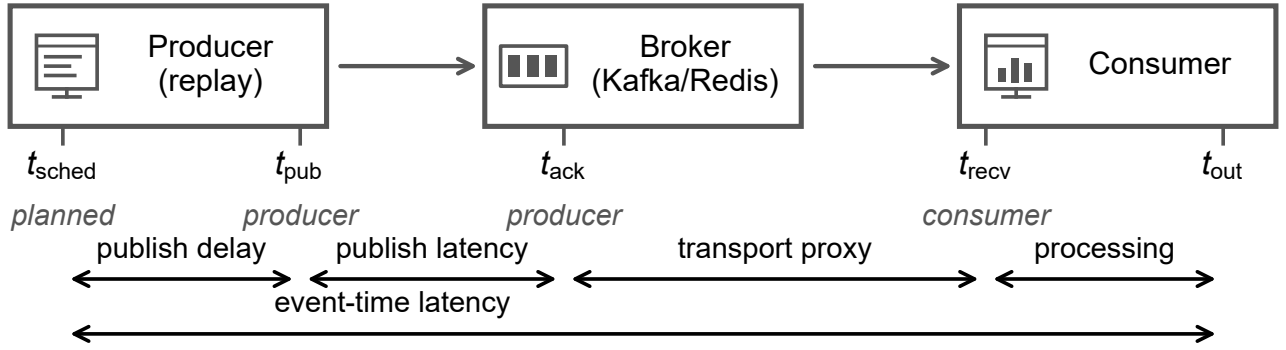


Fig. S1. **Five timestamps bracket a message's path.** A replay process paces recorded match events at a chosen rate into a broker, Kafka or Redis Streams, and a consumer reads them back. The upper four spans are the terms of Equation 2 and sum to the fifth, the event-time latency E .

S1.5. Where each timestamp is taken, and what each span covers

Of the spans the harness reports, only the transport proxy lacks a causal order between its two timestamps, and only it inverts (Table I). Figure S1 draws the four timestamps of Equation 1 and the fifth, t_{out} , at which Equation 2 closes the event-time latency. The publish delay, A and D each start from a timestamp taken before the event that ends them can happen, so all three are causal chains and a late thread can only lengthen them. S subtracts the acknowledgment's timestamp from the receipt's, both taken after the broker's append with no order between them, so it is negative whenever $A > D$ (Equation 3).

The two clients read the acknowledgment timestamp in different places, but in both it is read only after a thread that was not running has been woken and then given a core. The publish timestamp is read on the calling thread immediately before the client is invoked. Under Redis the acknowledgment timestamp is read on that same thread the moment the synchronous `xadd` returns, with no second thread involved. Under Kafka it is read, by default, in the client's I/O thread when the delivery callback fires. Our Kafka producer's `inline` mode instead timestamps on the calling thread once the publish future resolves, and requires one request outstanding.

The consumer reads t_{recv} when it holds the record, and an output timestamp t_{out} once it has handled the record. Our Kafka consumer parses the payload inside `poll()`, before t_{recv} . Our Redis consumer parses it between the two. The processing-time latency $t_{\text{out}} - t_{\text{recv}}$ is therefore 281 ns under Kafka and 19,480 ns under Redis, medians of run medians over 5,913 runs, and negative on none of the 738,730 events. The parse thus falls inside D for Kafka only, and Redis's median end-to-end latency of 800 μs leaves out a parse 2.4% as long. A longer D produces fewer negative spans, and Kafka already has the higher negative-span rate, so correcting the asymmetry by including that parse would widen the gap between the brokers in Table I. The event-time latency runs to t_{out} , not t_{recv} , so it contains both parses and is unaffected.

S2. THE SIGN CHECK, AND WHAT IT SELECTS

Moving the one-percent line changes how many runs the sign check rejects, but it changes no conclusion the paper draws. The rule is that of Section II-C. A run is rejected if more than 1% of its events carry a negative value in any latency component,

TABLE S6

MORE THAN HALF OF THE RUNS ON EACH TESTBED FAIL THE SIGN CHECK. RUNS BEHIND A REPORTED RESULT, BY TESTBED. A CONDITION IS USABLE ONLY IF ALL OF ITS RUNS SURVIVE. THIS CORPUS DIFFERS FROM THE SPAN RECOUNT OF TABLE I, WHICH COVERS EVERY CLOUD RUN THAT PRODUCED MATCHED EVENTS.

Testbed	Runs	Rejected	Conditions	Usable
Workstation	1,382	862 (62.4%)	76	8
Cloud	884	459 (51.9%)	40	13
Both	2,266	1,321 (58.3%)	116	21

TABLE S7

SELECTION BY THE SIGN CHECK DOES NOT DECIDE THE BROKER COMPARISON. (A) E1, WHOSE REJECTED RUNS' VALUES ARE LOST. (B) THE POWERED TRANSPORT CAMPAIGN AND ITS REPLICATION, WHOSE REJECTED RUNS' VALUES SURVIVE.

(a) E1

N	Redis kept	Shift (ms)	Worst case (ms)	Inside margin
1	8/8	+0.081	+0.081	yes
9	20/27	+0.021	-0.051	yes
10	17/30	+0.116	-0.485	yes
12	19/36	+0.053	-0.227	yes

Worst case: every rejected Redis run imputed at the largest value observed in its cell. Margin 1 ms.

(b) Powered transport campaigns

Campaign	N	Redis kept (%)	All runs	Runs kept	Change	V^* (ms)
powered	1	53.3	0.4095	0.4084	-0.0011	none
powered	9	43.7	0.4181	0.4156	-0.0025	0.60
powered	12	37.0	0.4197	0.4170	-0.0027	0.60
replication	1	25.0	0.3969	0.3807	-0.0162	0.55
replication	9	45.8	0.4120	0.4137	+0.0017	0.60
replication	12	39.8	0.4132	0.4157	+0.0025	0.55

All runs, runs kept: the Hodges–Lehmann shift, Kafka minus Redis, in ms. Change: kept minus all. V^* : the smallest value which, imputed to every rejected Redis run, would flip the shift's sign. The rejected runs' own medians center on 0.10–0.12 ms.

or if the median of any component is negative. A condition is usable only if all of its runs survive. We fixed the rule before the final campaign but after earlier campaigns had run, which is weaker than pre-registration, and the threshold sweep below exists partly to compensate. On each testbed the rule rejected more than half of the runs (Table S6).

S2.1. The threshold sweep

Per-run rates offer no natural threshold, so the line had to be chosen. On the workstation testbed 104 runs carry no negative span at all, and the rest spread over more than four decades of rate (Fig. S2a). Raising the threshold lowers the share of runs rejected (Fig. S2b), and at one percent the rule rejects 62.4% of the 1,382. The verdict on our first result does not change. Swept from 0 to 20%, the threshold makes none of that result's cells usable, since even the best of them keeps 23 of its 30 runs (S8).

S2.2. What the check selects, bounded

The check rejects unequally between configurations, so the corpus it leaves is not a random sample of the corpus measured. This is the most serious internal threat to any comparison drawn from the survivors. How we bound it depends on whether a corpus kept its rejected runs' values (Table S7). E1, the historical broker comparison on the cloud testbed, lost them. Panel (a) therefore imputes every rejected Redis run at the largest value observed in its own cell, and the worst-case Hodges–Lehmann shift stays inside the 1 ms margin in every cell. That bound holds only while the check keeps more than half of a cell's Redis runs. The cell with the smallest share kept sits 2.8 points above that line.

The powered transport campaigns, sized in advance for statistical power, fall below that line, and the broker equivalence of Section VI-D rests on them. The check keeps 25.0–53.3% of their Redis runs, under one half in five of the six cells. Their rejected runs' values survive, so panel (b) computes the selection directly. Restoring those runs moves the Hodges–Lehmann shift by at most 0.016 ms. Reversing its sign would take every one of them at 0.55–0.60 ms, about five times the 0.10–0.12 ms at which their own medians center. In the sixth cell no value reverses it.

A run that passes has cleared the cheapest available bar and nothing more. A latency inflated by load but still positive, for example, passes the check by construction (Section III-A).

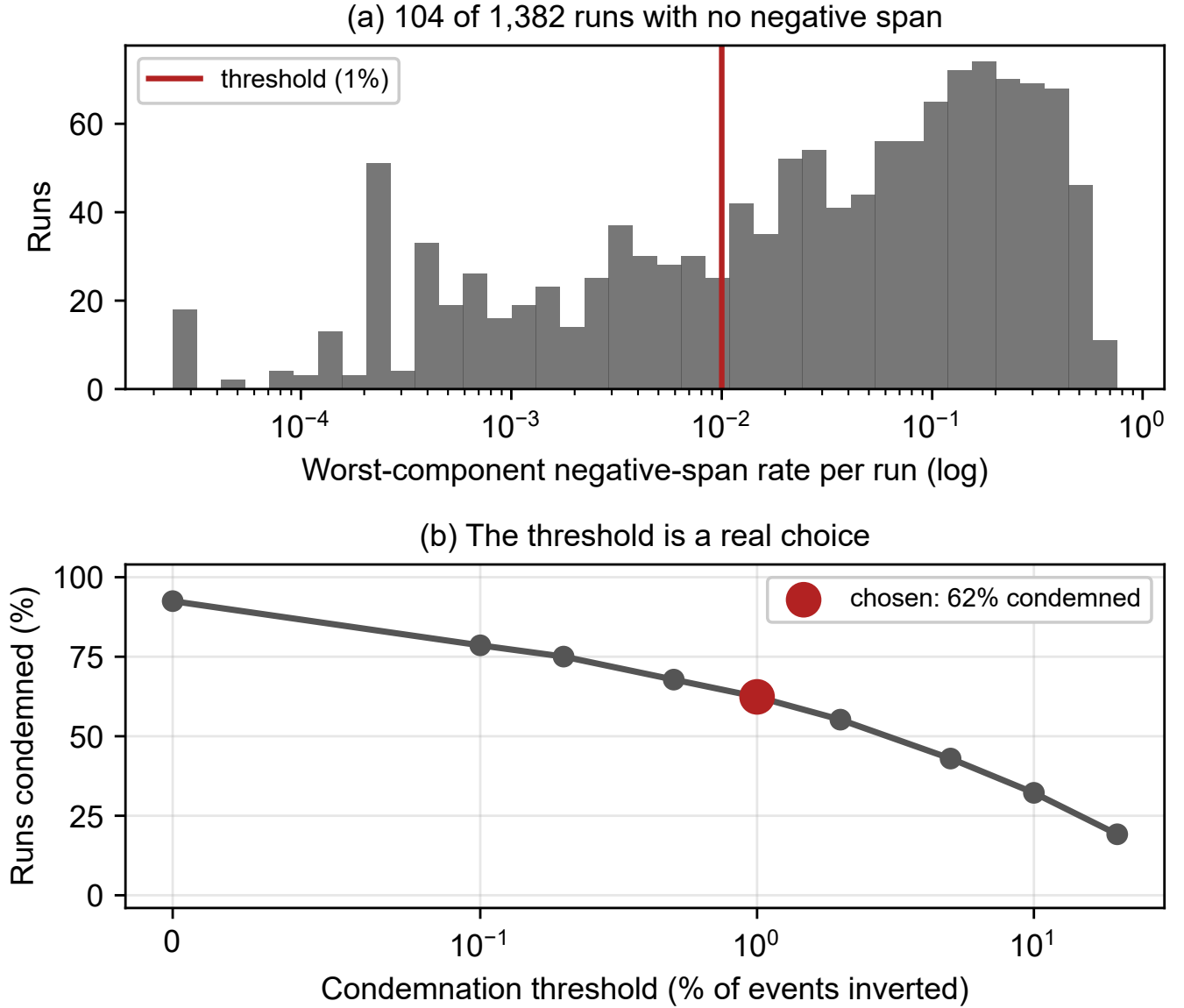


Fig. S2. **No natural threshold, and the choice moves no conclusion.** The sign check on the workstation testbed. (a) Per-run negative-span rates in the worst latency component: 104 runs carry none at all, and the rest spread over more than four decades. The one-percent rule passes more runs than those 104, and rejects 62.4% of the 1,382. (b) Share of runs rejected against the threshold, swept from 0 to 20% of events.

S3. FAILURE 1: THE FULL EVIDENCE

Failure 1, the negative latency, comes from the scheduler. Every manipulation moved the negative-span rate the way the mechanism predicts, whether it changed the timestamping thread's wait for a core or the end-to-end latency that the wait must outlast. A kernel trace that times the wait predicts the rate to within a third, unfitted. The evidence follows the order of Section IV: three manipulations, the model they support, the trace, the rival explanations and the timescale. The cost of timing from the acknowledgment, which the paper sets out among its practical implications (Section VI-B), comes last.

S3.1. Real-time priority cuts the rate in every matched pair

Moving the timestamping threads to real-time priority cut the negative-span rate between 7- and 80-fold in each of eight matched pairs, with utilization unchanged. Under `SCHED_FIFO` the threads preempt the background load instead of queueing behind it, so their occupancy falls while the machine stays as busy. If occupancy sets the rate, the rate should collapse toward its floor, that of a thread never preempted. Any account in which utilization alone sets the rate predicts no change, and the M/G/1 waiting-time form we had adopted and registered is such an account (S8).

The pairs come from three campaigns (Table S8). E-A5 is the original manipulation, E-A5b widened the range of loads to 60 and 95%, and E-A7 replicated E-A5. A rule fixed in advance withholds a pair whose two achieved utilizations differ by more than 0.05. It withheld 0 pairs, and the widest difference in the table, 0.002, is 25 times smaller than the tolerance.

TABLE S8

FACTOR OF 7 TO 80, WITH UTILIZATION MATCHED IN EVERY PAIR. NORMAL-PRIORITY AND REAL-TIME NEGATIVE-SPAN RATES AT THE SAME ACHIEVED UTILIZATION, AND THE FACTOR BETWEEN THEM, FROM THREE CAMPAIGNS. BOTH CONFIGURATIONS' ACHIEVED ρ IS PRINTED SO THAT THE MANIPULATION CAN BE CHECKED PAIR BY PAIR. GENERATED AT BUILD TIME FROM THE CAMPAIGN FILES.

campaign	load	ρ normal	ρ real-time	normal	real-time	factor
E-A5b	60%	0.6055	0.6035	0.0459	0.0017	27×
E-A5b	70%	0.7032	0.7025	0.1032	0.0144	7×
E-A5	75%	0.7531	0.7525	0.1320	0.0034	39×
E-A7	75%	0.7550	0.7537	0.1504	0.0044	35×
E-A5b	88%	0.8806	0.8809	0.2214	0.0034	66×
E-A5	88%	0.8809	0.8812	0.3049	0.0057	54×
E-A7	88%	0.8825	0.8825	0.2526	0.0054	47×
E-A5b	95%	0.9501	0.9501	0.2938	0.0037	80×

Only the normal-priority configuration moves with load (Table S8). The real-time configuration stays between 0.0017 and 0.0144 over the whole ladder, while the normal-priority one climbs with load from 0.046 to 0.30. That is what Equation S2 predicts when priority moves the occupancy p directly. The factor tends to grow with load but not at every step, since 60% gives 27× and 70% gives only 7×. It is therefore reported as a range. The real-time rates settle on both sides of the floor the model names in advance, $C_0 \approx 0.004$, which is therefore a scale and no bound.

Table II prints each factor without its interval. For the priority pairs the Katz 95% intervals are [21–74] at 75% load ($z = 19.8$) and [33–86] at 88% ($z = 31.9$). For the load geometry they are [1.80–2.39] in the original campaign ($z = 10.3$) and [1.73–2.43] in the replication ($z = 8.4$). The priority intervals are wide because the real-time configurations hold few negative spans. The Wilson intervals on the rates themselves are disjoint in every pair.

S3.2. One utilization, reached two ways, gives two rates

Utilization is not the variable, because one utilization reached two ways gives two rates. A concentrated load keeps k of the $C = 8$ cores fully busy and leaves the other $C - k$ free, while a spread load keeps every core busy part of the time and leaves none. If negative spans come from a timestamping thread that cannot find a free core, the spread load should produce more of them at equal utilization, and it does (Table S9).

At $k = 6$ both geometries sit at $\rho = 0.7531$, identical to four decimals, on one machine in one hour. Their rates differ 2.07-fold ($z = 10.3$ over 2,985 matched events per cell), which no function of ρ can do, since a function returns one value for one input. The replication E-A6b reproduces that cell almost exactly, at the same ρ and a factor of 2.05. It also reproduces the direction at $k = 5$. At $k = 7$ the difference is small, since one free core in eight is nearly as useless as none, and only the replication resolves it (1.19×, $z = 3.46$). Multi-server queueing already expects geometry to matter at fixed ρ , so the contrast refutes not queueing itself but the single-parameter form, in which ρ alone fixes the rate.

S3.3. A longer path inverts less often

A slower path gives a more reliable measurement, because the same stalls must outlast a longer interval to make the span negative. Padding the payload lengthens the path without touching the scheduler. In E-A10 the median of the transport proxy S rose 76.9-fold over 4 levels while the negative-span rate fell 4.1-fold [Katz 95%: 3.5–4.8], monotonically, and a repeat on another day agrees (Table S10). Serialization pushes utilization slightly up, which would raise the rate against the observed fall, so the effect is measured against its own confound. An account in which stress on the host causes the negatives predicts the opposite direction.

TABLE S9

AT $k = 6$ ONE UTILIZATION GIVES TWO RATES, IN TWO CAMPAIGNS. CONCENTRATED LOAD LEAVES $C - k$ CORES FREE; SPREAD LOAD DUTY-CYCLES ALL OF THEM AND LEAVES NONE. EVERY CELL IS 2,985 MATCHED EVENTS OVER 25 RUNS, AND z IS A TWO-PROPORTION TEST BETWEEN THE GEOMETRIES OF A ROW. ACHIEVED ρ AT $k = 6$ IS 0.7531 IN ALL FOUR CONFIGURATIONS; AT $k = 5$ IT IS 0.6284/0.6250 AND 0.6284/0.6259, AND AT $k = 7$ 0.8778/0.8809 AND 0.8778/0.8822.

k/C	E-A6 (original)			E-A6b (replication)			factors
	concentrated	spread	z	concentrated	spread	z	
5/8	0.0201	0.0379	4.09	0.0111	0.0509	8.89	1.88×, 4.61×
6/8	0.0824	0.1709	10.27	0.0600	0.1229	8.44	2.07×, 2.05×
7/8	0.2281	0.2415	1.22	0.1973	0.2342	3.46	1.06×, 1.19×

TABLE S10

SLOWER PATH, MORE RELIABLE MEASUREMENT, IN TWO CAMPAIGNS. PAYLOAD PADDING IN TWO CAMPAIGNS. THE MEDIAN OF S SPANS $76.9\times$ AND $76.8\times$ WHILE THE RATE FALLS $4.09\times$ AND $4.26\times$ $[3.62\text{--}5.01]$, BOTH MONOTONICALLY, WITH UTILIZATION HELD TO THE ρ SPAN SHOWN. EACH CELL IS 2,985 EVENTS.

Padding (B)	E-A10		E-A10b		ρ span
	Median S (ms)	Rate	Median S (ms)	Rate	
0	0.701	0.2603	0.761	0.2338	0.881–0.882
4,096	2.381	0.1906	2.474	0.1652	0.882–0.883
65,536	14.638	0.0888	16.059	0.0784	0.885–0.885
262,144	53.898	0.0637	58.482	0.0549	0.893–0.895

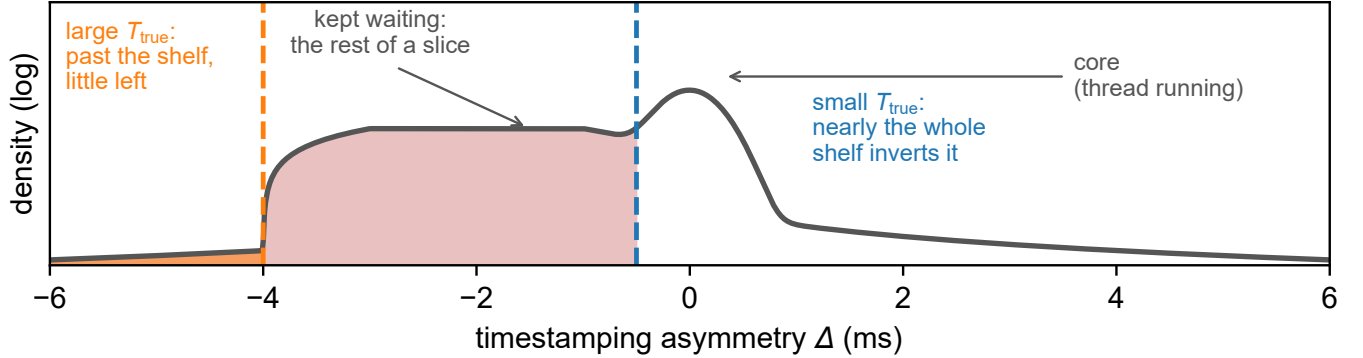


Fig. S3. **In the model, a short end-to-end latency is inverted by nearly the whole waiting state, and one longer than a slice and a tick by none of it.** The two-state model, drawn. The timestamping asymmetry Δ has a narrow core while the timestamping thread runs. While it waits, a shelf reaches from the core to -3 ms, the scheduler's base slice, and a tick beyond. The shape is schematic: two components drawn with the derived slice and tick, not fitted. The shelf, a wait for the rest of a slice, was read from the law campaign's recordings after its verdicts (S6.5). The measured distributions are the stall spectrum of Figure S4 and the per-acknowledgment waits of S6.5.

The fall is shallow. Lengthening the path 77-fold lowered the rate only 4.1-fold. Least squares of the log rate on the log median of S , which stands in for T_{true} , gives

$$\Pr[S < 0] = 0.238 T_{\text{true}}^{-0.339}, \quad R^2 = 0.990, \quad (\text{S1})$$

with a Student- t 95% interval on the exponent of 0.234–0.443 at 2 degrees of freedom. E-A10b returns 0.344 against 0.339, while the prefactor drifts (0.238 against 0.216). The four points of Table S10 describe the sweep and license no statement about the moments of the stall distribution (postmortem, S8). What they do show is the latency factor $G(T_{\text{true}})$ of Equation S2 as an experiment: the same stall distribution overlaps a short end-to-end latency almost entirely and a long one hardly at all, where a load-driven account predicts no change.

S3.4. Two states account for the rate, and the second is the run queue

Two states are the smallest model consistent with these data. A timestamping thread is either running, and timestamps late by a narrow jitter, or waiting for a core. Condition the identity of Equation 3 on the end-to-end latency. Write p for the thread's occupancy of the waiting state and G for the survival function of its residual stall there, the part of the wait still to come. To leading order,

$$\Pr[S < 0 | T_{\text{true}}] = p G(T_{\text{true}}). \quad (\text{S2})$$

Load moves p without fixing it (S3.2), so the form is written without a load term. Figure S3 draws it as the distribution of the timestamping asymmetry $\Delta = \delta_{\text{recv}} - \delta_{\text{ack}}$, the delay of the receipt's timestamp less that of the acknowledgment's, which turns negative while the acknowledgment's thread waits for a core.

By construction, the model makes negative spans cluster. Being preempted lasts an interval, so the events inside it are corrupted together. A Wald–Wolfowitz runs test on each run's sequence of span signs finds strong clustering at every load of E-A3, even with no background load (Table S11). Coarse kernel timestamps would corrupt events independently and give $z \approx 0$. From an idle host to the heaviest load before its processors are fully busy, E-A3's negative-span rate grows 61-fold while the jitter width grows only 5-fold. Two parameters move at very different rates. The jitter width σ_{core} belongs to the running state, and p to the scheduler. A scale family, in which load only stretches one distribution by changing its width, has one parameter, which is why it fails here. The highest rate, 0.37 at $\rho = 1$, is the largest over the conditions measured and not a bound, as the floor C_0 of S3.1 is a scale and not a bound.

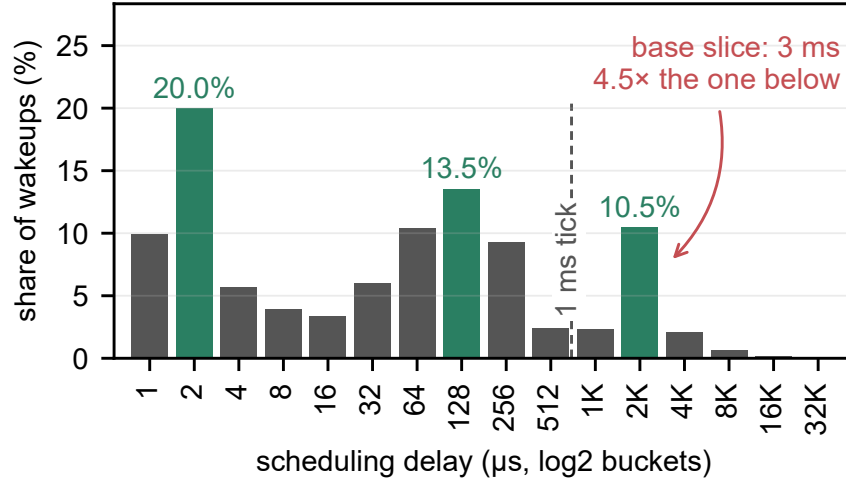


Fig. S4. **Traced scheduling delays are trimodal.** 551,956 wakeups, `bpfttrace` log₂ buckets, modes shaded. The jitter peaks in the 2–4 μs bucket, which alone holds 20.0% of all wakeups. A second mode at 128–256 μs holds 13.5%. The third, at 10.5%, sits in the bucket holding the scheduler’s derived base slice of 3 ms. The count of modes survives a 2× widening of the buckets in both phasings.

TABLE S11

SHORT OF FULL UTILIZATION THE RATE GROWS 61-FOLD AND THE JITTER ONLY 5-FOLD. E-A3, NINE LOAD LEVELS, 25 RUNS EACH, PRE-REGISTERED. THE JITTER WIDTH IS $\sigma_{\text{core}} = \text{IQR}/1.349$. NEGATIVE SPANS CLUSTER AT EVERY LOAD (RUNS-TEST z). THE THREE $\rho = 1$ ROWS ARE DISTINCT OVERSUBSCRIPTION LEVELS THAT THE UTILIZATION COORDINATE CANNOT SEPARATE.

Utilization ρ	Jitter width σ_{core} (ms)	Negative-span rate	Clustering z
0.003	0.126	0.004	−6.8
0.253	0.138	0.004	−6.8
0.504	0.180	0.006	−6.8
0.628	0.181	0.024	−6.9
0.753	0.238	0.076	−5.7
0.878	0.631	0.224	−4.9
1.000	1.224	0.371	−4.3
1.000	1.293	0.317	−4.7
1.000	1.710	0.261	−4.4

The model also predicts a shape. A scale family changes σ , so its tail curves rescale horizontally. The two-state model changes p , so its curves shift vertically and stay parallel on a log scale. The log ratio of two conditions’ tail masses should then not depend on the threshold. Over tail estimates backed by at least 20 events, that log ratio’s spread over thresholds has a median of 0.29 across conditions, a factor of 1.34, and a worst case of 0.91. The scale-family collapse fails by 23×. The rate therefore separates into p and G to first order, with the two largest spreads at intermediate load. The test was exploratory, and a confirmatory version was registered and run, the decisive campaign being the priority manipulation of S3.1.

On the load axis alone the model predicts nothing testable, since with p unconstrained any monotone curve in ρ can be written $p(\rho)G$. A fit on that axis cannot separate the load from the jitter width either. Restoring it to the form reaches $R^2 = 0.9905$, and freezing it at its mean does better, 0.9982, because the jitter width and ρ move together on this ladder. What the model does predict lies on the end-to-end latency axis, where S3.3 tests it.

The campaigns measure p only as a time average. E-A7 registered that real-time priority would cut the thread’s occupancy at least tenfold. Its scheduler counters show the occupancy falling 1.7- and 2.2-fold at the two loads, and the mean stall 3- to 5-fold, against a 35- to 47-fold fall in the rate. The registered prediction failed. Counters that report means cannot see a tail event, which is why the direct trace of S3.5 was needed. The campaigns therefore confirm the mechanism and its consequences and leave the factorization untested. Those consequences stand without it, because the direction under payload follows from Equation 3, the floor C_0 is the $p \rightarrow 0$ limit, and the trace of S3.5 compares two probabilities. The registered campaign later timed the waiting state for every acknowledgment and measured the shape of G , in a reading made after its verdicts (S6.5).

S3.5. A kernel trace predicts the rate unfitted, and changes it

The negative-span rate matches, to within a third, the probability that a run-queue stall outlasts the end-to-end latency. That probability is measured without reference to any negative span. With `bpfttrace` on the `sched_wakeup` and `sched_switch` tracepoints we recorded the delay between a timestamping thread becoming runnable and reaching a core, for every such transition in a run. The two quantities share no probe, no estimator and no data. One is a count of negative spans in application records, the other a kernel histogram of scheduler transitions. Across the three normal-priority configurations of Table S12 the

TABLE S12

KERNEL TRACING PREDICTS THE NEGATIVE-SPAN RATE, UNFITTED, IN EVERY CONFIGURATION THAT HAS ONE. $\Pr[\text{STALL} > 0.5 \text{ ms}]$ FROM THE `SCHED_WAKEUP` AND `SCHED_SWITCH` TRACEPOINTS, AGAINST THE NEGATIVE-SPAN RATE FROM APPLICATION TIMESTAMPS IN THE SAME CELL. EVERY CELL IS 2,985 EVENTS. EVERY TRACED REAL-TIME CONFIGURATION READS EXACTLY ZERO WHERE THE UNTRACED CONTROL READS 15 IN 2,985, SO THAT ZERO IS THE TRACER'S.

Campaign	Configuration	Load	ρ	Traced wakeups	$\Pr[\text{stall} > 0.5 \text{ ms}] / \text{rate}$	Ratio
E-A9	normal	88%	0.8822	551,956	0.181 / 0.231	0.78
E-A9	real-time	88%	0.8819	570,591	0.018 / 0.000	—
E-A9b	normal	75%	0.7543	709,819	0.168 / 0.128	1.32
E-A9b	real-time	75%	0.7531	641,308	0.016 / 0.000	—
E-A9b	normal	88%	0.8825	687,986	0.207 / 0.196	1.06 [†]
E-A9b	real-time	88%	0.8822	649,788	0.016 / 0.000	—
Untraced control: E-A9's first attempt, probe never attached						
—	normal	88%	0.8812	—	— / 0.272	—
—	real-time	88%	0.8809	—	— / 0.005	—

[†]Withheld by the tracer check, 28% drift against a 25% rule fixed in advance: shown, not used.

ratio of traced probability to measured rate is 0.78, 1.06, 1.32, with nothing fitted, and the tracer check below withholds one of the three. The ratios lie above and below one, so no bias smaller than a third can be resolved.

The tracer is not a free observer, and its cost was measured. A first attempt at the traced run, whose probe failed to attach, became an untraced twin. It ran two hours apart with the same design and measured 0.272 against the traced 0.231 ($z = 3.6$), the traced rate 14.8% lower. That gap lies inside the drift between campaigns. The normal-priority 88% configuration spans 0.221 to 0.305 over its four untraced measurements, and the replication's traced run reads 0.196. The tracer's cost affects the rate's level, on which no claim rests. Each prediction is held against the traced run's own rate, so the agreement above compares two quantities from one run, and a common perturbation moves both together.

The tracer check, a rule fixed in advance, decides which traced configurations are compared. One whose rate differs from its untraced comparator by more than 25% is not. The replication's 88% configuration measures 0.1956 against the control's 0.272, a drift of 28%, so its comparison is withheld. Its row stays in the table with the ratio it would have given, 1.06.

The tracer also erased the smallest rate. Every traced real-time configuration, three across two campaigns and two loads, recorded no negative span in 2,985 events, where the untraced twin of one of them recorded 15. At that rate the chance of one traced configuration showing none is 3×10^{-7} , and of all three below 10^{-17} . Attaching BPF to `sched_switch` therefore suppressed the negative spans in the real-time configuration. No claim rests on that configuration's level. The tracer changed the quantity it came to observe, and it did so where that quantity was smallest.

S3.6. Ten rival explanations, each excluded or bounded

No rival to the run queue survives the corpus, and four fall to the manipulations already shown (Section VIII). One clock excludes clock skew. Identical utilization at rates twofold apart excludes utilization. `SCHED_FIFO` excludes timer noise, which would delay a thread whatever its scheduling class, and a longer path that lowers the rate excludes stress. Coarse kernel timestamps fall to the runs test of S3.4. The other five need an argument each.

The second thread: Kafka's client timestamps the acknowledgment on its I/O thread when the delivery callback fires, and Redis's on the calling thread the moment `XADD` returns (Figure 1; S1.5). We built the harness expecting that hand-off to cause the negatives. The hand-off has a real effect. Timestamping Kafka on the calling thread shrinks the gap between the brokers' medians by about a quarter, 0.071 ms [95% bootstrap over runs: 0.039–0.113], and by 0.039 ms in a replication.

It is not the mechanism. Redis, which has no second thread, inverts on 8.19% of its events against Kafka's 8.67%, and on more of its runs (88% against 65%). Its calling thread, like any thread returning from a blocking socket read, must also be woken and scheduled before it reads the clock, and that wait delays its timestamp exactly as a callback thread's wait does. The cleaner control, Kafka's rate under both timestamping modes, cannot be computed, because that campaign kept only its medians.

The scheduler: The scheduler works as designed. A base slice with a wakeup-preemption check trades wakeup latency for throughput and fairness, and the escapes from it are opt-in: real-time priority, a preemption-free build, or a pinned busy-polling core. The kernel's `SO_TIMESTAMPING` already returns the time a packet arrived, and none of the tools we audited uses it.

The hypervisor: The hypervisor is bounded, not excluded, because both testbeds are virtualized. The cloud instances are virtual machines, and the workstation runs its brokers in Docker Desktop containers inside a WSL2 virtual machine on Microsoft's hypervisor. Two observations disagree with an account in which *steal time*, the time a virtual processor is ready to run but waits while the hypervisor runs another machine, causes the negatives. Real-time priority is a setting inside the virtual machine, which steal time would not yield to, and yet it cuts the rate 7- to 80-fold. The upper mode of the stall spectrum shares its bucket with the base slice that the virtual machine's own kernel derives (S3.7), not with any quantity the hypervisor exposes. Steal time may still contribute, and no testbed here can show how much.

The interpreter lock: The interpreter lock is present in principle and bounded in practice. The timestamping threads are CPython, which releases the lock around blocking I/O and retakes it on return, and the clock read is a Python call. A timestamp therefore waits for a core and then for the lock. The kernel trace cannot see the second wait, which happens in user space after the thread reaches a core. Priority cannot tell the two waits apart, since a `SCHED_FIFO` thread also reaches the lock sooner. Nor can their size, since the interpreter’s default switch interval and the base slice are both of millisecond order.

The direction of the disagreement settles it. A second wait in series with the run queue would make an estimator built from the run queue alone underestimate the observed rate. Its ratios, 0.78, 1.06, 1.32, lie above and below one instead. The registered campaign found the same failure in Kafka’s own Java client, which has no interpreter lock, at a lower rate (S6.3).

The clock: Clock incoherence across virtual CPUs [S3] is excluded twice. First, a measurement bounds the clocksource. The platform probe’s smallest observed increment of `clock_gettime(CLOCK_MONOTONIC)` was 90 ns, an upper bound on the cost of one read. That excludes `acpi_pm` by arithmetic, since its tick is 279 ns, and `hpet` by its published read cost near 1.2 μ s. What remains, `kvm-clock` or `tsc`, fits the bound, and each carries the coherence guarantee.

A `kvm-clock` that fast is on the `vDSO` path, which `kvmclock.c` enables only when the hypervisor sets `PVCLOCK_TSC_STABLE_BIT`, the flag that guarantees monotonic time across CPUs. A KVM guest selects `tsc` only when the kernel finds it constant, non-stop and stable, and abandons it if it later falls out of step between CPUs. Which clocksource ran is unrecorded, and the published read costs vary about twofold, against an order-of-magnitude margin.

Second, the corpus leaves no headroom. An offset between clocks large enough to produce the deepest negative span, -99.8 ms, would have driven the end-to-end latency D far below zero. The smallest end-to-end latency observed, 220 μ s, bounds any such offset 455-fold below what the deepest negative span needs.

S3.7. The upper stall mode survives coarsening, and its bucket holds the derived slice

The upper mode of the traced stall spectrum survives a change of bucket edges, and the bucket it occupies holds the scheduler’s derived base slice. How many local maxima a histogram shows depends on its bin edges as well as on its counts, and these edges came from `bpftrace`, as the millisecond came from the benchmark. The per-event stall durations were not retained, so coarsening is the only test available. Merging adjacent buckets doubles the bin width, and starting the merge at the first bucket or the second puts the edges in different places. The spectrum has 3 maxima on the tool’s own 16 buckets, 3 again under both phasings at 2 times the width, and fewer at 4 times, so the count holds to within a factor of 2 in bin width.

Above the upper mode the counts collapse. They fall 5.0-, 3.4- and 4.7-fold over the next three octaves and then 357-fold, each octave a doubling of the wait, whereas a power law would fall by the same factor in every octave. Power laws fitted below the mode and above it both fail a bootstrap goodness-of-fit ($p < 0.0004$ and $p < 0.0004$, over 2,500 replicates each).

The slice is derived from the kernel’s rule and the campaign’s own data. Each geometry condition loads k cores and records the achieved utilization, so k/ρ recovers 8 cores six times over, with a spread of 0.05. The kernel’s rule scales a 0.75 ms constant by one plus the integer base-two logarithm of the CPU count, up to eight CPUs, giving 4×0.75 ms = 3 ms [S4]. The constant is specific to this kernel and became 0.70 ms in v6.15, after our 6.8. The build sets `CONFIG_HZ = 1000`, a 1 ms tick.

With run-to-parity, a woken thread that loses the wakeup-preemption check waits for the running thread to finish its slice. The posting that gives the slice rule also notes that a task notices its deadline only at the next tick, so waits end in a band a tick wide. No scheduler tunable is written anywhere in the archived campaign, so the derived value is the one that applied. That is strong evidence, short of proof, and a future campaign should read `base_slice_ns` and `CONFIG_HZ` off the image under test. How long a woken thread actually waits was timed later by the registered campaign, which found that it waits out what is left of a slice, up to a whole one, in a reading made after its verdicts (S6.5).

S3.8. The proxy’s error falls as one over the end-to-end latency, and adding the publish latency back repairs it

The publish latency does not grow with the path, so the transport proxy’s relative error is A/D , and the shorter the path, the larger that error. Table S13 tabulates the curve of Figure 4 at nine path lengths, from the measured publish latencies. At the median publish latency the error passes 100% below 0.72 ms, and there an acknowledgment-timed harness stops measuring the path at all. The middle column is the one to read, because a publish latency is not a constant. A 10 ms path sits at 7% at the median publish latency, but a tenth of our conditions sit at 19% or more. A publish-timed harness reports the end-to-end latency itself and needs no correction from the table.

The two headline ratios of Section VI-B are medians across conditions of each condition’s median S divided by its median D , and they depend on that choice of estimator. A reader who differences the per-condition medians gets an understatement of $3.5\times$, and one who divides the pooled medians gets $2.6\times$. With the paper’s median of ratios, these are three defensible readings, a factor of 1.6 apart. The independence overshoot of Section III-A, how far assuming independence overpredicts the negative-span rate, is a median of ratios too, taken within runs above a 0.1% floor, where the ratio of medians nearly agrees with it ($7.0\times$ against $7.4\times$).

The understatement is a median, and half the conditions are worse. Its interquartile range runs 4.0 – $7.5\times$, and the worst condition reaches $14\times$. Two identical systems whose clients timestamp in different places differ by the difference of their

TABLE S13

BELOW 0.72 MS THE MEDIAN PUBLISH LATENCY EXCEEDS THE END-TO-END LATENCY IT CORRECTS. COMPUTED FROM THE MEASURED PUBLISH LATENCIES. LEFT: THE ERROR A/T_{true} ON ONE REPORTED LATENCY AT THE CORPUS'S MEDIAN PUBLISH LATENCY, 725 μs . MIDDLE: THE SAME ERROR AT THE TENTH AND NINETIETH PERCENTILES OF THE PUBLISH LATENCY ACROSS CONDITIONS, 500 AND 1900 μs . RIGHT: THE GAP A COMPARISON WOULD REPORT BETWEEN TWO IDENTICAL SYSTEMS WHOSE CLIENTS DIFFER ONLY IN WHERE THE ACKNOWLEDGMENT IS TIMESTAMPED, A CALLBACK THREAD AGAINST AN INLINE TIMESTAMP. WHERE IT READS "INVERTS" THE APPARENT GAP HAS CHANGED SIGN. ON EVERY ROW BELOW THOSE NOTHING IS NEGATIVE, AND THE SIGN CHECK FIRES ON NONE OF IT.

True path T_{true}	Error on one measurement	p10–p90 across conditions	Apparent gap between two identical systems
0.25 ms	290%	200–760%	inverts
0.5 ms	145%	100–380%	inverts
1 ms	72%	50–190%	inverts
2 ms	36%	25–95%	67%
5 ms	14%	10–38%	22%
10 ms	7%	5–19%	11%
25 ms	3%	2–8%	4%
50 ms	1%	1–4%	2%
100 ms	0.7%	0.5–2%	1%

TABLE S14

ADDING THE PUBLISH LATENCY BACK RECOVERS THE END-TO-END LATENCY WHERE THE CHECK REJECTS AS WELL AS WHERE IT PASSES. RECOVERY ERROR AS A PERCENTAGE OF EACH CONDITION'S MEDIAN END-TO-END LATENCY. Q3 IS THE UPPER QUARTILE. EXACT IS THE SHARE OF CONDITIONS RECOVERED TO THE MICROSECOND THE CAMPAIGN RECORDS, WITH ITS WILSON 95% INTERVAL.

Conditions	n	Median	Q3	Worst	Exact (Wilson 95%)
Check accepts	42	3.4%	20%	36%	33% [21–48]
Check rejects	28	12.2%	19%	36%	21% [10–40]

publish latencies, an apparent gap of 11% at an end-to-end latency of 10 ms, with the sign check firing at neither. Over the 24 matched workloads of Section VI-B the proxy inflates the gap between the brokers 1.7-fold, with an interquartile range of 1.2–2.0. On 21% of the pairs it narrows the gap instead.

The repair is Equation 1 rearranged. The identity holds per event, and the producer measures A on the clock the consumer reads, so a harness that records its own publish latency recovers the end-to-end latency it failed to measure. Table S14 gives the error of that recovery, as a percentage of each condition's median end-to-end latency, split by the sign check's verdict.

Nothing in these data separates the two populations. The Hodges–Lehmann shift over all 1,176 pairwise differences, rejected minus accepted, is 1.2 points [95% percentile bootstrap: -1.8 to 8.3]. The worst condition on each side has the same error, and the upper quartiles are within a point and a half. The medians differ because 33% of the accepted conditions recover the end-to-end latency exactly, and a pile at zero drags a median down without shifting the rest of the population. The pile is not itself a detectable difference. The exact shares differ by 11.9 points, but their Newcombe 95% interval, -10.0 to 30.7 , includes zero. Without the exact recoveries the medians are 17.1% over 28 accepted conditions and 14.3% over 22 rejected ones, and the shift between them is 0.0 points [95% bootstrap: -8.0 to 7.1].

The shapes differ to the eye (Figure S5). Counted in three bands of the error, exact, below 20% and 20% or more, the accepted conditions divide 14, 14 and 14. The rejected ones divide 6, 17 and 5, a smaller pile at zero with most of the rest below the edge. The edge is the accepted population's upper quartile, and the accepted conditions split into thirds for any edge above 14.3% up to it. The distributional test does not separate them either. A tie-corrected two-sample Kolmogorov–Smirnov statistic of 0.27 has $p = 0.12$ over 20,000 permutations.

The data support no equivalence either. The 90% bootstrap intervals on the two shifts would establish it only for margins as wide as 8.3 points over all conditions and 7.1 over the non-zero ones, about half the non-zero median errors. The claim is therefore the weaker one, that the repair holds where the check rejects as well as where it passes. The median is also no bound. For a quarter of the accepted conditions the error reaches 20%, and for one it reaches 36%.

S3.9. Five objections, each answered in one line

None of the five objections in Table S15 moves a number in the main text, and each has a one-sentence answer. The table gives that answer and says where the full one is. The interpreter lock, a sixth objection, is answered in Section VIII and in full in S3.6.

S3.10. Timing from the publish call keeps the receiving thread's wait for a core, which three measurements put under a millisecond on the mean

A latency timed from the publish call carries the receiving thread's wait for a core. By Equation 1, $D = S + A$ for every message, so the acknowledging thread's wait cancels: it lengthens A by as much as it shortens S . The receiving thread's wait

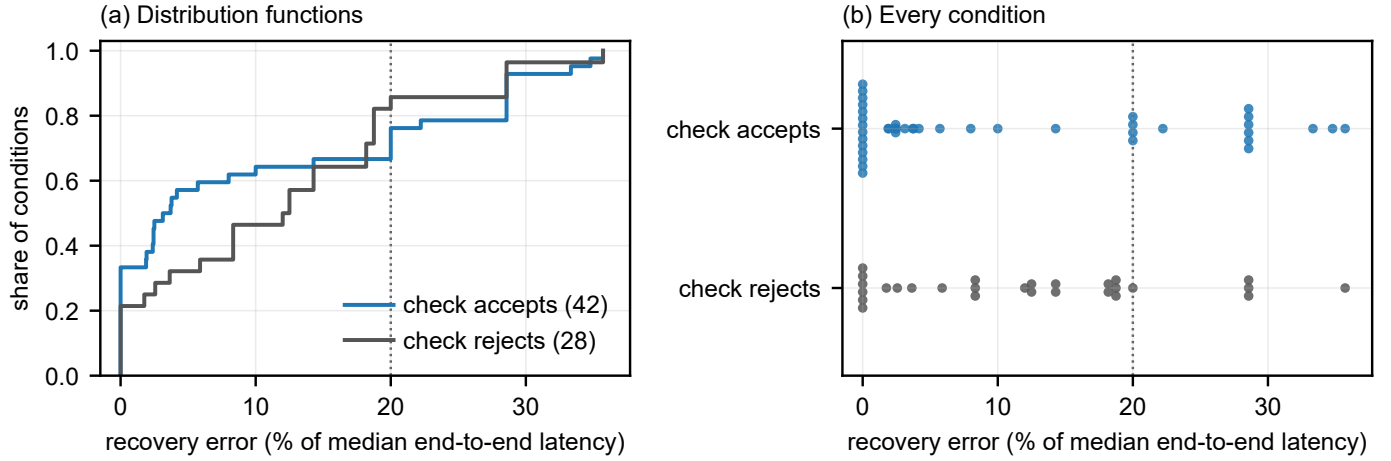


Fig. S5. **Two populations with no detectable shift between them.** (a) Empirical distribution functions of the recovery error over the 42 conditions the check accepts and the 28 it rejects. The dotted line is the band edge, the accepted population’s upper quartile of 20%. The curves open with steps of 33% and 21% at zero, change order at 18.2% and change back at 34.8%, where each population has a single condition left above. (b) Every condition, one mark each.

TABLE S15

NONE OF FIVE OBJECTIONS MOVES A NUMBER IN THE MAIN TEXT. ONE ROW PER OBJECTION. THE LAST COLUMN SAYS WHERE THE FULL ANSWER IS, IN THIS SUPPLEMENT OR IN THE POSTMORTEM.

Objection	Answer	Full answer
Is a negative span RFC 1242’s negative latency?	No: RFC 1242’s negative latency [S5] is a cut-through forwarding artifact, while a negative span is two host timestamps subtracted with one of them written late (Equation 3).	Section IV-A
Does retention mean Kafka’s log-retention setting?	No: retention here is the fraction of the samples a tool takes that survive into what it reports, a property of the measurement.	Section II
Why the wall clock and not <code>CLOCK_MONOTONIC</code> , and is the TSC coherent across virtual CPUs?	The wall clock gives two hosts a common epoch, the ordering a monotonic clock would guarantee on one host is shown by the data, and the probe admits only clocksources the kernel uses where cross-CPU coherence is asserted.	S3.6; postmortem, S15 and S16.6
Is the two-host harness’s sample count a repeated figure?	No: the two-host campaign was matched run for run to the one-clock campaign, so both hold 905,040 samples, and its 0 negatives are its own count.	S4.6; postmortem, S22.1
Did the kernel tracer change the rate it was used to explain?	Yes, from 0.272 untraced to 0.231 traced ($z = 3.6$), so each prediction is held against the traced run’s own rate, and the real-time configurations’ zeros are the tracer’s.	S3.5

does not cancel. The consumer reads its clock only after its thread has been woken and given a core, so every D holds that wait, which is never negative and which the sign check of Section II-C cannot see. After the registered campaign we measured it three ways in the same runs, by a plan and four predictions written before any run.

The runs used the first x86 pair, both brokers, 75 and 88% load and the base slice at 3 ms, with no delay added to the network. Eight setups, each broker at each load with the consumer at normal or at real-time priority, ran in six shuffled rounds. Three runs added before anything was read gave every setup at least three traced runs, and all 51 runs passed the integrity rule. Method 1 runs the consumer alone in the real-time class at priority 80 and measures how much that lowers D at unchanged load. Method 2 traces the scheduler in a random half of the runs and measures the receiving thread’s own wait for a core before each timestamp, as S3.5 did for the acknowledging thread. Method 3 captures the receiver’s packets in every run with the kernel’s nanosecond timestamps, and times each message from the packet that carries it to the consumer’s timestamp. That delay includes the wake-up, the wait and the client’s own work.

The three estimates run from 0.41 to 0.94 ms (Table S16) and lie within a factor of 1.5 of each other at three of the four points. The wait lives in the tail. At normal priority the median message waited 18–46 μ s and the mean 0.51–0.92 ms, while the ninetieth percentile reached 2.13–2.92 ms. Real-time priority brought the mean wait down to 10–11 μ s. On Redis it cut the ninetieth percentile of D to under 1 ms and left the median almost where it was.

The first prediction held. On both brokers at both loads the wait’s mean at normal priority lay between 0.3 and 1.2 ms and its ninetieth percentile above 1.5 ms, and real-time priority took the mean below 0.1 ms. The other three failed, each on Kafka. There real-time priority lowered the ninetieth percentile of D by less than the 1 ms predicted at 75% load, and moved the median by more than the 0.3 ms allowed at 88%. Method 3’s delay was at least as long as the traced wait for 99.7% or more of every traced run’s messages, against the 95% predicted. But Kafka’s client then worked longer than the 0.3 ms predicted before its timestamp, where Redis’s stayed under it. At 75% load Kafka’s first two estimates lie further apart than the factor

TABLE S16

THREE MEASUREMENTS PUT THE RECEIVING THREAD’S WAIT AT 0.41–0.94 MS OF THE MEAN LATENCY. ONE ROW FOR EACH BROKER AND LOAD. METHOD 1 IS THE FALL IN MEAN D WHEN THE CONSUMER ALONE RUNS AT REAL-TIME PRIORITY. METHODS 2 AND 3 ARE THE FALLS, FROM NORMAL TO REAL-TIME PRIORITY, IN THE MEAN TRACED WAIT AND IN THE MEAN DELAY FROM THE KERNEL’S RECEIVE TIMESTAMP TO THE TIMESTAMP. EACH FIGURE IS A SETUP’S MEDIAN OVER ITS RUNS. THE CLIENT’S WORK IS THE MEDIAN, OVER TRACED RUNS, OF THE DELAY BEYOND THE TRACED WAIT.

Broker, load	Added to the mean of D (ms)			Largest over smallest	D , normal $\rightarrow$ real-time priority (ms)		Client’s work (ms)
	Method 1	Method 2	Method 3		Median	Ninetieth percentile	
Kafka, 75%	0.43	0.70	0.64	1.64	2.03 $\rightarrow$ 1.82	5.21 $\rightarrow$ 4.27	0.37
Kafka, 88%	0.94	0.91	0.92	1.04	2.52 $\rightarrow$ 1.92	5.66 $\rightarrow$ 4.53	0.39
Redis, 75%	0.47	0.50	0.47	1.07	0.89 $\rightarrow$ 0.84	3.06 $\rightarrow$ 0.93	0.11
Redis, 88%	0.43	0.62	0.41	1.49	0.87 $\rightarrow$ 0.83	2.86 $\rightarrow$ 0.89	0.12

of 1.5 allows. We have not tested why Kafka differs.

Three things qualify the figures. The tracer is not a free observer, as S3.5 found: at normal priority the traced runs’ mean D ran 5–22% above the untraced runs’, so the second method may read the wait somewhat high. In five runs a message arrived more than 150 ms late, three of them during stalls of the whole driver, the kind S6.4 traces to the receiving side’s disk. A setup’s value is a median over its runs, which sets them aside. And three of the four setups at real-time priority hold only two untraced runs, too few to judge the tracer there. The plan, the register of every run, the reading and its record are in the repository under docs/results/recv_wait, read by scripts/recv_wait.py.

S4. FAILURE 2: THE FULL EVIDENCE

The benchmark’s printed latency describes only the samples its publish schedule lets through the positivity filter, and each link in that claim can be checked against a source line, a counter or a registered prediction. Every benchmark run below is in embedded mode, with the whole benchmark in one JVM.

S4.1. Two source lines make the filter, and counting what they drop changes no printed number

What the positivity filter drops can be counted without changing the filter. In LocalWorker.java:294 the benchmark computes its end-to-end span as

```
MILLISECONDS.toMicros(now - publishTimestamp)
```

from two millisecond timestamps, so the value, though in microseconds, is a whole number of milliseconds. In WorkerStats.java:95 the positivity filter,

```
if (endToEndLatencyMicros > 0)
```

decides whether that value reaches the histogram. The message has already been counted as received, but nothing counts what the condition drops.

We add counters in an else branch of that condition and change nothing else: not the latency computation, not the filter, not any reported statistic. The benchmark’s output is therefore exactly what it would have been unpatched, and its printed summary can be set beside the retention behind it. The four counters, for zero, negative, most negative and kept samples, separate sign. A negative span and a sub-millisecond end-to-end latency both fail a > 0 test, and one unsigned total cannot tell them apart (S4.5).

The counts are exact. The counters are shared across the JVM and their totals printed once at shutdown. A run that fails discards nothing, and its zero count would look measured, so a validity gate writes it as invalid, with its reason and no count. Unlike the benchmark’s histograms, our counters include the one-minute warmup, yet a matched sweep with the warmup disabled leaves retention unchanged. The campaign totals 223 instrumented runs, swept one axis at a time with background load, message size, producer rate and warmup otherwise fixed.

S4.2. The phase of the publish schedule sets retention, as a 1970 counter note already showed

Where each publish instant falls within a step of the clock decides whether its sample survives. An end-to-end latency shorter than the step τ survives only if it crosses a grid instant, where the timestamp changes value. Otherwise both timestamps round down, or floor, to the same value, and the positivity filter deletes the zero difference (Fig. S6a). When phases are uniform, retention is $\theta = T_{\text{true}}/\tau$, which is Equation 6 of the paper.

A paced producer does not draw its phases uniformly. If its publish interval T_{pub} satisfies $T_{\text{pub}}/\tau = a/q$ in lowest terms, it visits only q distinct phases, so its retention is a whole count out of q (Fig. S6b). Call the retentions $0, 1/q, \dots, 1$ the vertices of the configuration’s q -lattice, and the interval between neighboring vertices, $100/q$ percentage points wide, a lattice step. Replicates of one configuration land on the vertices that bracket θ , and their spread obeys

$$\text{spread} \longrightarrow \begin{cases} 100/q & \text{when } \theta \text{ sits mid-step,} \\ 0 & \text{when } \theta \text{ sits on a vertex.} \end{cases} \quad (\text{S3})$$

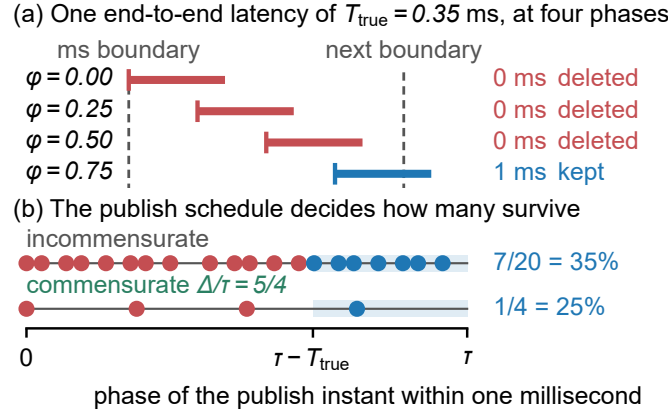


Fig. S6. **Where a publish instant falls within a millisecond decides whether its sample survives.** (a) One end-to-end latency of $T_{\text{true}} = 0.35$ ms at four phases of one millisecond. Three finish inside it, compute to zero and are removed. The fourth crosses the boundary and is recorded as one whole millisecond. (b) A sample survives from phase $\tau - T_{\text{true}}$ onward, the shaded band, so uniform phases retain T_{true}/τ . A commensurate producer visits only q phases and retains a count out of q . The end-to-end latency drawn is off-vertex by choice, since at 0.5 ms the two answers coincide.

The step width is thus an upper bound on the spread, reached only when replicates realize both bracketing vertices. Retention at the incommensurate rates puts T_{true} at 0.49 ms, half a millisecond to within replicate noise (Section V-E). One half is a vertex of every q -lattice with even q and of none with odd q , so the spread rule predicts full step width for odd denominators and almost none for even ones.

None of this arithmetic is new. Hewlett-Packard’s Application Note 162-1 [S6] carries it for a counter timing an interval, point by point.

- The retention identity. For an interval of $Q + F$ clock periods the note gives $\Pr[R = Q+1] = F$. The share of readings on the upper of the two straddling counts is the fractional part. That is Equation 6.
- The class of a repetition rate. The class is the denominator of the ratio of repetition rate to clock rate in lowest terms, our q , and it fixes how many distinct readings the counter can produce.
- The ceiling on averaging. Averaging N readings can improve resolution by at most a factor $1/q$, not $1/\sqrt{N}$. The note names $q = 1$ as the corner where averaging cannot help at all, and a paced producer reaches it whenever its interval is a whole number of clock steps (Table S19).
- The symptom. A reading appears to hang on a round number while the input is known to vary.
- The cure. Phase jitter on the repetition rate, so the samples stop landing on the same points of the clock’s cycle. Hewlett-Packard patented it. US 3,938,042, filed in 1974 and granted in 1976, varies the phase of the counter’s own time base “to destroy coherence between the clock signal and an unknown signal” [S7]. A benchmark’s time base belongs to the operating system, so the benchmark must dither the publish instant instead.

The metrology of analog-to-digital converters standardizes the same phase arithmetic. IEEE Std 1241-2010 requires the record of its histogram test to hold a number of input cycles “relatively prime” to the number of samples (§6.4.1), and names the practice coherent sampling (§5.4.1) [S8]. Neither source contains the deletion. The counter keeps both readings, and the retained fraction is its estimator.

S4.3. Retention sits on the vertices in nine of the ten configurations able to show it

The phase account predicts each configuration’s behavior from its denominator alone, and every configuration a test can judge sits nearer its q -lattice than a continuum of retentions would. Publish latency cannot be what moves retention. It takes two values, 0.3 and 0.4 ms, across settings whose retention varies 279-fold. The correlation it does have (Spearman +0.31; Fisher 95%: +0.08 to +0.51; 71 settings) is positive, as the retention law predicts.

The spread rule holds in 10 of the 12 commensurate configurations (Table S17). A spread above half the step width counts as full, so the test needs no tolerance. The predictions split ten full against two flat. The two misses, 700 and 1250 msg/s, are predicted full and measured flat.

Spread is a range over replicates that can each land on only two vertices, so a configuration shows its full width only if its replicates land on both. At $q = 5$ and $q = 7$ the upper vertex is the less likely of the two, so five replicates can all fall on the lower one even when the lattice holds exactly. Spread cannot tell that outcome from a refutation.

We built a replacement statistic for that reason after the campaign had run. It was not registered, and its first results were committed in the same change as its code.

For each configuration, $\bar{d}$ is the mean distance of its replicates to the nearest vertex of its q -lattice, divided by the step width. The null is a continuum, in which replicates are drawn from a Normal distribution centered on the retention the incommensurate

TABLE S17

REPLICATE SPREAD FOLLOWS THE PHASE DENOMINATOR q IN 10 OF THE 12 COMMENSURATE CONFIGURATIONS. PAYLOAD, LOAD, DURATION AND HOST ARE FIXED. THE POSITION IS THE DISTANCE FROM THE MEASURED CONTINUOUS VALUE TO THE NEAREST VERTEX, AGAINST HALF THE STEP WIDTH. A CONFIGURATION IS PREDICTED FULL WHERE ITS POSITION EXCEEDS A HALF, AND SHOWS FULL WHERE ITS SPREAD EXCEEDS HALF THE STEP WIDTH. THE DAGGERED ROWS ARE THE TWO MISSES, RESOLVED IN TABLE S18. INCOMMENSURATE CONFIGURATIONS HAVE NO LATTICE STEP. THEIR SPREADS OF ABOUT THREE POINTS ARE THE NOISE FLOOR.

rate	T_{pub}/τ	q	n	width	pos.	predicts	spread	shows
1000/s	1/1	1	5	100.0	0.98	full	99.4	full
500/s	2/1	1	5	100.0	0.98	full	99.5	full
250/s	4/1	1	5	100.0	0.98	full	98.7	full
400/s	5/2	2	5	50.0	0.04	flat	17.6	flat
600/s	5/3	3	6	33.3	0.95	full	32.7	full
300/s	10/3	3	10	33.3	0.95	full	31.2	full
800/s	5/4	4	5	25.0	0.07	flat	7.2	flat
1250/s [†]	4/5	5	5	20.0	0.91	full	7.6	flat
625/s	8/5	5	10	20.0	0.91	full	19.3	full
875/s	8/7	7	5	14.3	0.87	full	10.7	full
700/s [†]	10/7	7	5	14.3	0.87	full	6.5	flat
900/s	10/9	9	5	11.1	0.84	full	6.9	full
<i>incommensurate — phase set effectively continuous</i>								
333/s	1000/333	>64	5	—	—	≈ 0	4.5	—
383/s	1000/383	>64	5	—	—	≈ 0	2.8	—
457/s	1000/457	>64	5	—	—	≈ 0	2.1	—
611/s	1000/611	>64	5	—	—	≈ 0	3.1	—
889/s	1000/889	>64	5	—	—	≈ 0	1.7	—
1053/s	1000/1053	>64	4	—	—	≈ 0	3.0	—
1219/s	1000/1219	>64	4	—	—	≈ 0	4.5	—

TABLE S18

RETENTION SITS ON THE q -LATTICE IN NINE OF THE TEN POWERED CONFIGURATIONS, TESTED AGAINST A CONTINUUM NULL. HERE $\bar{d}$ IS THE MEAN NORMALIZED DISTANCE TO THE NEAREST VERTEX, AND THE NULL COLUMN IS ITS MEAN UNDER THE SIMULATED NULL. THE COLUMN p_{Holm} IS A MONTE CARLO p -VALUE (4,000 DRAWS), HOLM-CORRECTED WITHIN ALL 12 CONFIGURATIONS; IT DECIDES THE VERDICT. FLAT (COINCIDENT) MARKS CONFIGURATIONS WHERE THE TWO HYPOTHESES PREDICT ALIKE AND THE TEST HAS NO POWER.

rate	a/q	n	$\bar{d}$	null	p_{Holm}	verdict
1250/s	4/5	5	0.131	0.351	0.003	grid
1000/s	1/1	5	0.003	0.479	0.003	grid
900/s	10/9	5	0.265	0.340	0.131	not resolved
875/s	8/7	5	0.209	0.382	0.003	grid
800/s	5/4	5	0.104	0.058	1.000	flat (coincident)
700/s	10/7	5	0.329	0.416	0.009	grid
625/s	8/5	10	0.112	0.447	0.003	grid
600/s	5/3	6	0.034	0.469	0.003	grid
500/s	2/1	5	0.043	0.490	0.003	grid
400/s	5/2	5	0.093	0.020	1.000	flat (coincident)
300/s	10/3	10	0.221	0.468	0.003	grid
250/s	4/1	5	0.181	0.489	0.003	grid

rates imply at the configuration's rate, with a measured standard deviation. Neither parameter is fitted to the configuration under test. The one-sided p -value comes from 4,000 Monte Carlo draws, which puts its floor at 2.5×10^{-4} before correction. It simulates the null at the observed number of replicates rather than leaning on an asymptotic distribution, which matters at five.

Of the 12 configurations, ten are powered, meaning the null and the lattice predict different distances. Of these, nine reject the continuum null at $\alpha = 0.05$ after Holm correction, eight of them at the Monte Carlo floor (Table S18, Fig. S7). The tenth, 900 msg/s, rejects before correction ($p = 0.044$) and not after ($p = 0.131$), so we report it as unresolved. The two unpowered configurations, 400 and 800 msg/s, have lattices that pass through θ , where the two hypotheses coincide. The test reports them as undecidable. Both configurations the spread rule missed reject the continuum: 700 msg/s at 0.009 and 1250 msg/s at 0.003.

Two rates that share a denominator land on the same vertices. The registered control is 600 msg/s, an interval of 5/3 ms and so the same $q = 3$ as 300 msg/s, registered in advance to land near 33%. Its replicates were 33.98, 34.29, 34.90, 36.17, 65.86 and 66.67: four on the 1/3 vertex and two on the 2/3 vertex, with nothing between. Their median, 35.5%, is near the 33% registered, and none left the thirds lattice, the registered falsifier. The upper two sit on the 2/3 vertex that 300 msg/s, half the rate, also reaches. Table S19 shows the $q = 1$ corner replicate by replicate.

The registered test that manipulates the physics is the payload flip. Payload moves T_{true} , and so moves θ within its lattice step, with nothing else changed. Measured at an incommensurate rate (Table S20), 32 KB puts θ almost on the 2/3 vertex of the $q = 3$ lattice and 64 KB puts it mid-step. The prediction was that this one manipulation would move a 300 msg/s configuration onto a vertex and off it again.

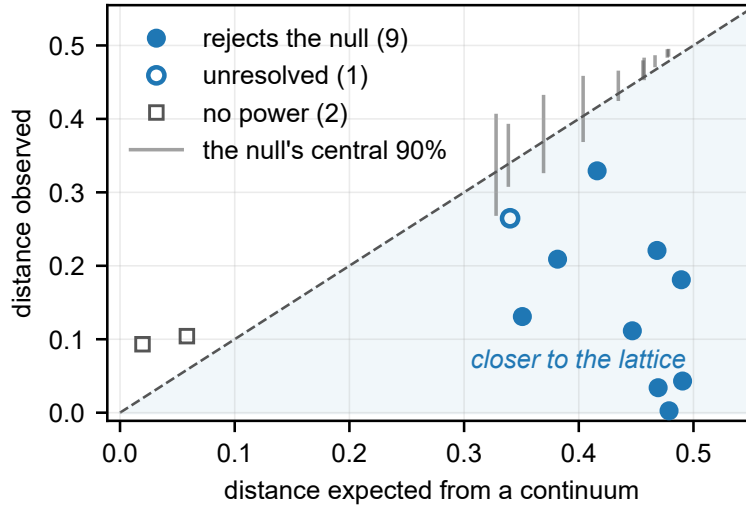


Fig. S7. **All ten powered configurations fall on the lattice side of a continuum, and nine survive correction.** Observed mean distance to the nearest vertex against the distance a continuum would give, the diagonal. Each gray bar is that configuration’s simulated null, uncorrected, and a marker below its bar rejects it. Filled circles also survive Holm correction. The open circle, 900 msg/s, sits below its bar, 0.2647 against 0.2680, but not after correction ($p = 0.131$). The two squares have no power and no bar.

TABLE S19

ONLY THE COMMENSURATE RATE IS UNSTABLE. RETENTION AGAINST THE PRODUCER’S PUBLISH INTERVAL, WITH MESSAGE SIZE, BACKGROUND LOAD, DURATION AND HOST FIXED. AT 500 MSG/S THE INTERVAL IS EXACTLY TWO MILLISECONDS, SO $q = 1$. THE TWO INCOMMENSURATE RATES AGREE WITH EACH OTHER ALTHOUGH THEIR INTERVALS DIFFER BY 19%. PREDICTIONS WERE RECORDED BEFORE THE RUNS.

Rate	Interval	Multiple of 1 ms?	Retention per replicate	Spread
500/s	2.000 ms	exact	0.47, 1.51, 1.55, 18.02, 99.99	99.5
457/s	2.188 ms	no	48.77, 49.50, 49.69, 50.51, 50.87	2.1
383/s	2.611 ms	no	50.28, 51.09, 51.45, 51.77, 53.13	2.8

The spread flipped from full to flat and back, as predicted (Fig. S8). At 32 KB the five replicates spanned 62.06–75.69%, a spread of 13.6 points, with three agreeing to 0.06. At 64 KB they spanned 73.06–99.78%, a spread of 26.7 points. Both cross the flat–full boundary at half the step width, 16.7 points, in the predicted direction. One registered detail failed. The 32 KB configuration pins at 69.2%, not on the vertex at 66.67%.

The flip was one of six predictions registered with falsifiers before its campaign. The flip was confirmed, three predictions were refuted by their falsifiers, one could not be evaluated, and one split (postmortem, S1.7).

S4.4. A deleted zero changes what the distribution is conditioned on

Deleting the samples that compute to zero conditions the reported distribution on being positive, leaving only the samples above zero, so the retained fraction is part of the measurement. Two classical sources come close to this argument. The Guide to the Expression of Uncertainty in Measurement (GUM) models a digital reading of resolution δx as a stimulus that “can lie with equal probability anywhere in the interval $X - \delta x/2$ to $X + \delta x/2$ ” (Clause F.2.2.1) [S9]. That rectangular model, with

TABLE S20

AT A RATE INCOMMENSURATE WITH THE MILLISECOND, RETENTION READS OFF THE END-TO-END LATENCY. RETENTION AGAINST PAYLOAD AT 457 MSG/S, WHERE SAMPLES STAY DEPHASED. THE IMPLIED T_{true} INVERTS EQUATION 6. UP TO 8 KB THE ADDED DELAY IS COMPARABLE TO THE REPLICATE NOISE; THE 32 AND 64 KB LEVELS CARRY THE RESULT.

Payload	Retention (3 reps)	Median	Implied T_{true}
200 B	51.08, 52.03, 52.88	52.03%	0.520 ms
1 KB	50.11, 50.32, 51.47	50.32%	0.503 ms
2 KB	52.13, 54.62, 54.77	54.62%	0.546 ms
4 KB	51.86, 54.17, 56.24	54.17%	0.542 ms
8 KB	54.92, 55.22, 57.89	55.22%	0.552 ms
32 KB	68.01, 68.44, 69.40	68.44%	0.684 ms
64 KB	84.21, 85.36, 86.82	85.36%	0.854 ms

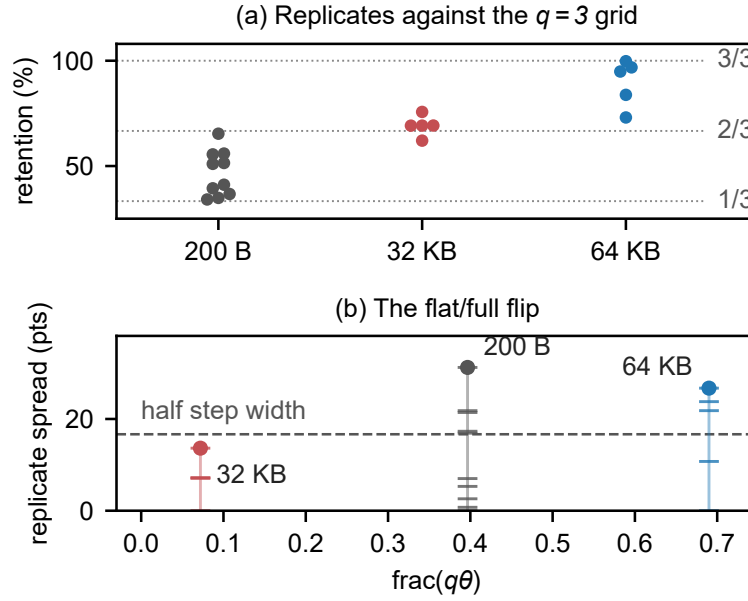


Fig. S8. **One manipulated variable moves a configuration onto its lattice and off it.** (a) Replicates at 300 msg/s ($q = 3$) by payload, against the thirds lattice: 200 B straddles both branches, 32 KB pins near the 2/3 vertex, and 64 KB re-opens to the upper step. (b) Replicate spread against $\text{frac}(q\theta)$, each stick showing the replicates its spread is made of, with θ each configuration's own mean retention in this campaign. The registered prediction placed 32 and 64 KB at 0.055 and 0.56 from the payload sweep's θ . The flat–full boundary at half the step width is crossed in both directions.

its variance $(\delta x)^2/12$, holds for retained readings. The positivity filter removes one side of the interval, so what remains is asymmetric and has neither the rectangular shape nor that variance.

Pratt showed that discarding zero differences before ranking breaks a natural monotonicity. A sample not judged significantly positive can become so when every observation is decreased [S10]. His procedure ranks all observations, zeros included, drops only the zeros' ranks, and refers the statistic to its null distribution conditional on the number of zeros. Our claim is weaker and more practical, that the zero count must be reported so a reader can see what the distribution is conditioned on. The exact parallel is an interpretation Pratt reports from Meier, that a zero arises because “with only our crude measurement, we do not know” the sign a finer measurement would resolve. That is the sub-resolution sample of Failure 2, stated in 1959.

Applied to the population of Table I (Figure S9), a positivity filter on millisecond timestamps removes 338,242 of the 738,730 events, 45.8% (95% bootstrap interval, resampling runs: 45.4–46.2). Of those, 303,468 computed to exactly zero and 34,774 to less, and a harness that filters this way counts none of them. Of the five dispositions in panel (c), two shrink the sample and report no count of what was removed, two keep the sample size by reporting a value never measured, and one keeps the data. This is the filter applied to our own spans. What the benchmark's own instrumented runs retained is the other measurement, in Section V-D of the paper.

S4.5. Two floored clocks disagree by exactly one millisecond, so the sign of a discard names the failed clock

The sign of a discarded sample says which clock failed, and the benchmark's two drivers show both kinds. The Kafka driver's 214 runs discarded 10,913,263 samples, and not one was negative. The Redis-driver replication's 9 runs carry all 41,403 of the campaign's negatives, every one exactly $-1000 \mu\text{s}$. A single run holds 41,397 of them, half of all the samples it took, beside six kept.

The drivers explain the difference. The Redis driver takes its publish timestamp from `entry.getID().getTime()` in `RedisBenchmarkConsumer.java:72`, the stream-entry identifier the Redis server assigns on XADD. The Kafka driver uses a producer-set timestamp. So the Redis span subtracts a server-side timestamp from a consumer-side clock reading, across two hosts, with both floored to the millisecond. The Kafka span floors two timestamps whose difference is taken inside one JVM.

Two consequences follow. First, two clocks floored to the same resolution and offset by less than one millisecond can disagree by one millisecond and by no other amount. The only negative a floored cross-host difference can produce is therefore $-1000 \mu\text{s}$, and every one of the corpus's 41,403 negatives is that value. Clock synchronization was clean at every logged minute, so this failure needs no misbehaving clock, only two clocks whose readings are floored. Second, a difference taken inside one process has no second clock to disagree with, which is why the Kafka discards hold no negative. The same positivity filter thus hides a resolution failure under Kafka and a clock failure under Redis, and an unsigned discard counter could not tell them apart. The two drivers also do not measure the same interval: client to client on one clock under Kafka, server to client across two under Redis. The benchmark does not flag the difference.

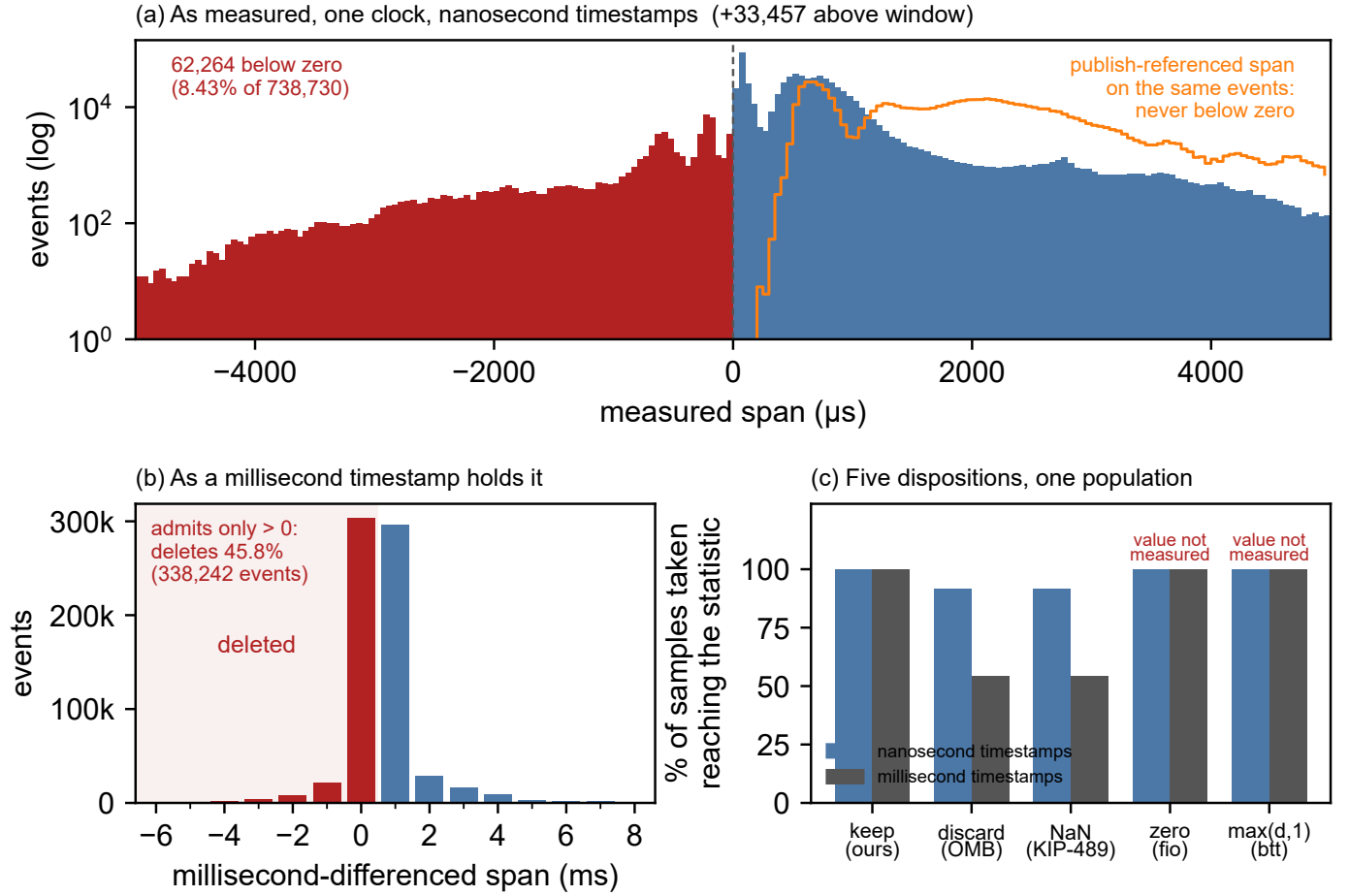


Fig. S9. **Positivity filter on millisecond timestamps would delete 45.8% of our corpus, uncounted.** (a) At nanosecond resolution the acknowledgment-timed span goes below zero 62,264 times in 738,730; the publish-timed span, on the same events and clock, never does. (b) Differenced millisecond timestamps land on 0 or 1 by where they fall against the millisecond boundaries alone, and the filter deletes every value at or below zero. (c) Five dispositions applied to the same population.

S4.6. An independent harness reproduces the deletion, and across two hosts the loss follows the clocks

The deletion does not depend on the benchmark's code, and across two hosts the deleted fraction drifts with the clocks. We reproduced the positivity filter in a Python harness that shares no code with the benchmark, with the same timestamp, subtraction and filter. On one clock it recorded 0 negative differences across 905,040 samples in 12 runs, as a causal chain requires, and the grid reappeared intact. Across two hosts with disciplined clocks, 12 runs again recorded 0 negatives in 905,040 samples and an intact grid, but the deleted fraction followed the drift of the clock offset. Cross-host retention wandered from 13.4 to 26.9% over four replicates at 457 msg/s, the incommensurate configuration. Of the three rates, it is the only one without a lattice spread wide enough to hide the drift. On one clock the same configuration's replicates spanned 0.98 points. Nothing records either the synchronization state or the fraction the filter deletes.

The benchmark's own distributed mode, where its subtraction spans two clocks, could not be measured. It never completed a run in eleven attempts: eight at three background-load levels, including none, and three more run with full diagnostics. The unloaded attempts failed like the loaded ones, which refutes our own hypothesis that processor starvation was the cause. The three diagnosed attempts failed identically, in the coordinator-to-worker path before any workload started. The first attempt, made before the validity gate of S4.1 existed, wrote a vacuous zero that the artifact keeps marked invalid. The gate refused every later attempt, each with its reason. The distributed mode's exposure to the deletion therefore rests on reading its source. The timestamp is written on the producer's host and read on the consumer's, and the positivity filter drops every difference at or below zero.

S4.7. A patched benchmark still reads the brokers' millisecond timestamps

Repairing the benchmark does not reach the brokers' own timestamps, because both brokers write them in milliseconds. Kafka's `record.timestamp()` is a millisecond field on the wire. Anything built on the field inherits a millisecond resolution, unless it carries its own finer timestamp in the payload. Documenting that, or exposing a finer timestamp attached to the record,

TABLE S21

TEN TOOLS, READ AT SOURCE. THE SPAN EACH MEASURES, THE CLOCK IT READS, WHAT HAPPENS TO A VALUE AT OR BELOW ZERO, AND WHETHER THE TOOL COUNTS WHAT IT DISCARDS; ONE ROW PER SPAN WHERE A TOOL MEASURES TWO, AND A DASH WHERE NOTHING IS DISCARDED. THE SOURCE LINE BEHIND EVERY ROW IS IN THE ARTIFACT’S REGISTRY.

Tool	Span measured	Clock	What happens to the value	Counts
Apache Kafka (bundled)	end-to-end	nanosecond	truncate to the resolution	no
Apache Kafka (bundled)	publish	millisecond	truncate to the resolution	no
Apache Pulsar perf	end-to-end	millisecond	filter, ≥ 0	no
NATS CLI	end-to-end	nanosecond	refused by own library	no
OpenMessaging Bench.	end-to-end	millisecond	filter, > 0 , truncate to the resolution	no
OpenMessaging Bench.	publish	nanosecond	—	—
RabbitMQ PerfTest	end-to-end	nanosecond	substitute a value	no
Rezulus	scheduler wakeup	nanosecond	discard	yes
blktrace/btt	block stage	nanosecond	substitute a value	no
emqtt-bench	end-to-end	millisecond	—	—
fio	I/O completion	nanosecond	substitute a value	no
wrk2	end-to-end	microsecond	refused by own library	no

is the change to ask of Kafka. The Redis `XADD` entry identifier embeds a server-clock time floored to the millisecond. One documented sentence saying it is not comparable with a client clock would have prevented the driver defect of S4.5. No layer here misbehaves. The benchmark is the only party that assumes something none of them promised: that observing an event tells you when the event happened. For benchmark authors the checks are those of Table III, and one more: never subtract a server-assigned timestamp from a client clock reading without saying so.

S5. THE TOOL REGISTRY

Seven of the ten tools we read at source dispose of a sample at or below zero, or shorter than the resolution, without counting it. Every verdict traces to a line of source a reader can check. Each row of the registry file carries one source line, the file and symbol it came from, the upstream URL and the date it was read.

The class labels are not stored. They are recomputed on every build and every test run by the classifier that read the OpenMessaging Benchmark a month before the other tools were audited. A reader who disagrees with a verdict can therefore check the line against the pattern that matched it. The classifier sees only what its patterns recognize. They were written against JVM sources and had to be extended before they could read an Erlang guard or clock call, so the registry’s count of disposals is a lower bound for the field.

We call a tool’s own conditional on the sign of an interval its *guard*. A guard is a deletion only if what it guards is the sample itself, not a counter. Table S21 is the registry, one row per span because one tool can treat two spans differently, generated from the registry file at build time. Its sources, in row order, are [S11], [S11]–[S14], [S14]–[S20].

The seven silent disposals take three forms. A filter drops what its comparison excludes. The test > 0 deletes every sample that computes to exactly zero, while ≥ 0 keeps those and drops only negative spans. A filter shrinks the sample the statistic is computed from and reports no count of what was removed. The benchmark filters at > 0 . Apache Pulsar’s performance client guards at ≥ 0 , so it drops negative spans and keeps the zeros the benchmark deletes, which then pile up visibly at zero.

A substitution keeps the sample count, so any retention computed downstream reads 100%, which makes it harder to detect than a filter. A library refusal leaves the sample unrecorded because the tool never checks whether its library rejected it. Truncation is a separate loss: it keeps every sample but loses precision. The benchmark truncates before it filters, and Kafka’s bundled tools only truncate. The notes below carry what the table cannot.

- The OpenMessaging Benchmark appears twice because it treats its two spans differently (Section V-C).
- The `EndToEndLatency` tool bundled with Kafka cannot invert, because it uses one thread, one monotonic clock and a synchronous `publish` call. It loses the measurement anyway. Each sample prints in floating point, but the `long[]` its percentiles come from is filled by the integer division `elapsed / 1000 / 1000`. On a sub-millisecond path its p50, p99 and p999 therefore all read zero. The companion `ProducerPerformance` takes its publish-to-acknowledgment difference on `System.currentTimeMillis()` and casts it to `int`.
- In `fio` the guard appears in three timing functions, each returning zero for a negative interval under the comment “time warp bug on some kernels?”. The behavior is defensive and reasonable in isolation. What is missing is any record that it fired.
- In `btt`, `tdelta()` returns one nanosecond whenever the stage timestamps are out of order, with no comment, no counter and no mention in the manual page.
- The `wrk2` load generator detects the negative interval, prints a block that begins “We are about to crash and die”, and records the value anyway without checking the result. It builds against its own copy of `hdr_histogram.c`, whose `assert()`s are live in the shipped build. That copy returns `false` for an out-of-range index, a refusal the tool never checks. Either the assertion aborts the run or the refusal drops the sample, and nothing counts the loss.

- In `emqtt-bench` the `> 0` guard protects a counter and leaves the histogram alone, so nothing is deleted from the sample.
- RabbitMQ `PerfTest` hands its difference on with no sign test, but the method that computes it, `getDifference`, returns its absolute value, so a negative span enters its metrics as a positive one of the same size, uncounted. Our first reading stopped at the call site and filed `PerfTest` with the tools that pass a negative on. The law campaign’s documentation audit read the method, and the registry now carries both lines.
- The NATS CLI appends every difference to the list its minimum, median and maximum are read from, negatives included, and is exposed to clock skew between hosts. Its percentile table is recorded into `hdrhistogram-go`, whose bound check refuses the index a negative value maps to. The CLI ignores the error, so its percentile table disagrees with its minimum, median and maximum, and nothing counts the difference. Our first reading stopped at the append. The documentation audit read the histogram.
- Rezolus counts its discards in a metric that names the cause.
- Two further tools do not make the measurement. `SPECjms2007` defines Delivery Time and never reports it, and Redpanda’s `rpk` ships no latency benchmark.

The same disposal has reached the broker itself. In November 2025 Kafka’s coordinator metrics were found throwing on negative durations after a host clock stepped backwards, and the merged fix clamps the negative to zero with no counter and no log line [S21]. That is silent substitution, adopted by Kafka itself. Its cause, a wall clock that genuinely stepped backwards, is a different mechanism from Failure 1. The same project makes the opposite choice a few directories away. Kafka Streams’ end-to-end latency metric records the wall clock less the record timestamp with no filter, so a record timestamped in the future is recorded as a negative latency [S22]. One project clamps the negative in one metric and keeps it in another, so the disposal is a choice.

The positivity filter persists downstream, in forks, and upstream, in the benchmark’s own repository. Of the 40 readable public forks of the benchmark surveyed on 2026-08-24, 40 carry the filter’s expression unchanged. That limits how many forks can have changed the expression, and says nothing about why. Confluent’s fork, one of the vendor distributions practitioners run, was archived by its owner on 28 February 2026 and is read-only, with 60 forks of its own (read 2026-09-12). A live fork may drop the filter tomorrow. An archived one cannot, and neither can anything downstream of it. Upstream, a search of the benchmark’s issue tracker for the variable the filter tests, run on 2026-09-13, returns 1 item: pull request #56, which introduced the filter [S23]. No issue or pull request has named it in the eight years since. That is why the checks of Table III are addressed to benchmark authors and to their readers.

Other runtimes document the structure behind Failure 1, a timestamp written when a thread next runs. The `librdkafka` client library serves its delivery-report callback from `rd_kafka_poll()` and the other queue-serving calls [S24]. The callback therefore runs on the application’s own thread, at whatever moment that thread next polls, rather than when the broker’s acknowledgment arrives. The gap between the two plays the part of δ_{ack} in Figure 1. The `fio` documentation defines completion latency for asynchronous ioengines as running to “the time ... when the I/O’s completion was reaped by `fio`” [S25], so a reaping timestamp is written into the definition of the metric. Neither frames this as a hazard. The property is public, and its consequence for a subtraction across two processes is drawn nowhere we could find. An origin timestamp written by a thread that must first be woken is late by that wait, so the span it opens turns negative whenever the wait outlasts the end-to-end latency.

Reading a tool tells what it does with a negative value; running it tells whether it ever meets one. The registered law campaign ran eleven benchmarking tools on the first x86 pair, four rounds each, under predictions fixed from their source before any run (its tools block, frozen with the plan; the amendments that touched it are in Table S23). Table S22 lists them. Under a delay added in steps, every tool’s slope lies in the band fixed in advance, and ten tools were judged to time on one clock in every run that decided it. The one tool spanning two clocks kept every value, negatives included, in each round that decided it. The per-run verdicts are in `docs/results/tools/tools_t1.csv` and `tools_t2.csv`, folded by `scripts/tools_block.py` from the runs, and the write-up with its five recorded faults is `docs/results/tools/t1-t4-eleven-tools.md`.

S5.1. Few published results carry the deletion’s signature, and none states what it kept

Published results rarely carry the deletion’s signature, and no report we found says how many samples it kept. The audit behind that sentence was registered, committed and public before any report was read. It fixed the signature, the places to search, the rule for inclusion, the coding and the outcomes, and nothing in it changed afterwards.

The primary signature is a printed end-to-end median of exactly 1 ms, the smallest value the benchmark can print and the value every retained sample takes when the true end-to-end latency is shorter than the resolution. The strong signature adds a p99 of exactly 1 ms. Neither says a published number is wrong. Each says the number came from a fraction of the samples, a fraction the report does not state.

The search covered Google Scholar, Semantic Scholar, the web, the engineering blogs of the vendors whose brokers the benchmark has drivers for, and the benchmark’s repository and its public copies, for reports dated up to 28 September 2026. A report was included if it named the benchmark or a fork of it as its tool and printed at least one numeric end-to-end percentile.

TABLE S22

ELEVEN TOOLS, RUN. THE REGISTERED CAMPAIGN’S TOOLS BLOCK, FOUR ROUNDS PER TOOL ON ONE PAIR OF MACHINES. INTERVAL: WHAT THE TOOL TIMES, AS ITS INVOCATION WAS READ BEFORE ANY RUN. SLOPE: HOW FAR ITS FIGURE MOVED PER MILLISECOND OF DELAY, DIVIDED BY THE NUMBER OF TIMES THE INTERVAL CROSSES THE DELAY, IN ITS LOWEST AND HIGHEST ROUND. CLOCKS: HOW MANY CLOCKS ITS TWO TIMESTAMPS COME FROM, AS MOVING THE TOOL’S CLOCK BACK BY MORE THAN THE INTERVAL IT TIMES SHOWED. RUNS: THE OFFSET RUNS THAT DECIDED THIS, OF ALL OF THAT TOOL’S. THE REST WERE UNDECIDED.

Tool	Interval	Slope	Clocks	Runs
hey	round trip	1.00–1.04	one	8/8
k6	round trip	1.00–1.02	one	8/8
memtier	round trip	0.96–1.00	one	7/8
Valkey benchmark	round trip	0.98–1.05	one	8/8
Vegeta	round trip	0.99–1.00	one	8/8
wrk2	round trip	0.92–1.04	one	4/8
Kafka producer	to ack.	1.00–1.14	one	8/8
Kafka end-to-end	one process	1.01–1.02	one	6/8
NATS latency	one process	0.99–1.06	one	7/8
RabbitMQ PerfTest	one process	0.97–1.00	one	6/8
librdkafka perf.	two processes	0.96–1.05	two	6/8

The audit included 43 reports, printing 1,100 configurations, of which 868 print a median. The primary signature occurs in eight configurations in two reports, both public repository text. Five are Redis rates in a notebook behind a 2023 journal paper that shows its medians only in a chart, and three are maximum-rate runs in three result files of a 2021 fork. The strong signature occurs nowhere, and no included report states its retention or a discard count.

The rules did not foresee several patterns. 14 of the included reports print medians below a millisecond, which the original benchmark’s clock cannot produce, among them vendors’ copies whose maintainers changed how the span is timed. One 2024 fork prints p95, p99 and p999 all at exactly 1 ms beside an average just above it and no median column, so by the registered rule it does not count. Four configurations print a median of zero, from empty histograms. The coding, the search log and every excluded report with its reason are in the artifact (postmortem, S25.1).

S6. THE REGISTERED LAW CAMPAIGN

A law for the negative-span rate of Failure 1, fixed before its runs, had one prediction confirmed on every campaign, three confirmed on one broker only, and the rest not confirmed. What held on every campaign that tested it, five on Kafka across the three pairs, is that load raises the plateau. No test of whether the fall follows a slice set by hand was confirmed. Frozen programs judged every prediction over 8,102 runs on machines the earlier campaigns never ran on. The paper gives the outcome in Section VIII, and the postmortem, Part V, keeps the run-by-run record.

S6.1. The law, and the design that tested it

The registered law says that the scheduler’s settings alone fix where the negative-span rate falls as the end-to-end latency lengthens, and how wide the fall is. It assumes that when a timestamping thread is kept waiting, it waits out a whole slice s of another task, and then up to one tick h . The rate then sits on a plateau $c + p$ while the end-to-end latency D is shorter than the slice, falls across one tick, and rests on a floor c beyond it:

$$r(D) = c + p \cdot \begin{cases} 1, & D \leq s, \\ (s + h - D)/h, & s < D < s + h, \\ 0, & D \geq s + h. \end{cases} \quad (\text{S4})$$

Here p , how often the timestamping thread is kept waiting, is the occupancy of Equation S2, and c is the floor that ordinary short delays leave. The slice and the tick predict where the fall starts and how wide it is. The plateau’s height and the floor are measured. In the plan, a reading is negative when the thread that timestamps the acknowledgment waits longer than the margin $D - g$. The margin is a run’s median end-to-end latency less its median publish latency g , the plan’s letter for A .

Three pairs of Microsoft Azure machines in one region ran the campaign, each a driver with eight virtual CPUs and a broker with two; two pairs have AMD EPYC 9V74 processors and one has Arm Neoverse N2. The plan’s blocks A1 to A9 are written L1 to L9 here. L1 sets the slice on the first x86 pair, and L4 on the Arm pair. L2 sets the tick, on three kernels built from one source tree that differ in the tick alone: 1, 4 and 10 ms. L3 sets the load, L5 the number of online CPUs, L7 real-time priority for the timestamping threads, and L8 the client, Kafka’s own Java client against our Python one. L6 records every `python3` thread’s waits in half of L1’s and L3’s runs, and L9 records each of those threads’ scheduling events in half its own runs. M0 asks why, when messages come in bursts, the end-to-end latency does not grow one-for-one with the added delay.

Each block carries numbered predictions, judged in Table S24. P1 and P9 place the fall at the slice, on the first x86 pair and on the Arm pair. P2, P2b and P2c tie its width to the tick, and P2d keeps its start at the slice at every tick. P3a and P3b

ask what load does to the fall and to the plateau, P4 what real-time priority does, P7 whether the slice and the fall follow the core count, and P8 whether Kafka's Java client behaves like our Python one. P6 and L9's three predictions ask whether recorded waits predict the rate.

Each run replays messages at a steady 50 a second, or a football match's bursts in M0, and measures its end-to-end latency, publish latency and negative-span rate after a 30 s warm-up. The end-to-end latency is set by delaying only the traffic to the receiver's own network address, so the acknowledgment's path is untouched. Each session first calibrates how far each added delay moves the end-to-end latency. An end-to-end latency shorter than the client's own with nothing added cannot be set, and was not run.

The integrity rule judges every run as it ends. A run counts when every condition it was meant to have took effect, from the load within 3 points of its target to the added delay within $50\ \mu\text{s}$ of its setting. Otherwise it is repeated, up to three attempts. A stopping rule checked the publish latency, and a run whose median publish latency moved too far stopped its campaign. In its final form, from version 20, the stopping rule compared a run with its setup's earlier runs in the campaign. The plan's own name for the publish latency, and for the stopping rule on it, is "got-it".

A round runs every setup once. Each campaign that tests a prediction took its rounds, at least four and at most forty, from simulating its own decision rule on data that follow the law. L2 and L9, which have no rule the simulation can take, ran at the floor of four, M0 ran a fixed ten, and no campaign ran more than 24. A prediction about where the fall sits is read twice: on a free reading, a curve required only never to rise, and on the law's fitted shape. It passes only where both agree. The judging read 8,102 runs in 156 campaigns, taken from 16 to 27 September 2026, and 7,996 of them counted.

S6.2. Every change to the plan, and what prompted it

The plan changed after results came in, and every change says whether a result prompted it. It was first frozen as version 5 on 16 September 2026, before its first run, and the version in force at the end is the thirty-second, freeze 28. No frozen version is ever edited. A change is a new, numbered freeze, public before the runs it covers, whose first section lists what changed, why, and whether a result already seen prompted it. Table S23 gives every freeze with that answer.

Versions 6 to 8 corrected what stage 0's first runs found in the machines, the code and the plan's own checks, before any campaign that tests a prediction began. P3 became P3a and P3b in version 9, before any load run was read. Three kinds of change followed the campaigns' own results. Version 14 judged P3a only where the plateau reaches 2%. The stopping rule was corrected five times, each time after it had stopped a campaign. And three versions corrected programs that had already judged, each stating what the verdicts it touches had been.

Blocks added as the campaign ran, in versions 24 and 27 to 29, were each read by a program fixed before their runs. The last three of those versions followed a check of all the data on 25 September. P6 was given its program in version 30, at a margin no run had been read against.

S6.3. Load raised the plateau everywhere, and the fall did not sit where the law put it

Load raised the plateau in all five load campaigns, all on Kafka, with every rise's interval above zero. Table S24 gives each verdict as its frozen program returned it. On Kafka, real-time priority took the plateau to zero in every run, and the fall followed the core count at the one count whose slice is at least Kafka's shortest end-to-end latency. Kafka's own Java client shows the same fall and the same effect of priority, at a plateau a third to a quarter of the Python client's.

On the first x86 pair the free reading placed the fall's halfway point inside its band at every slice each broker reaches, four for Kafka and five for Redis. The fitted shape did so at two of four slices for Kafka and two of five for Redis. On the Arm pair the free reading held on both brokers and the fitted shape failed on both (Figure S10). P1 and P9 need both readings, so neither is confirmed. Load moved the fall toward shorter end-to-end latencies in every campaign, and P3a held in none.

S6.4. Aberrant runs are marked, not removed, and one verdict moves without them

The plan keeps every run, and re-runs none whose conditions held. A program reads a mark for each aberration from the records, and an aberrant run carries its mark wherever it is drawn. Five marks keep a run out of every judge. One marks void runs, set aside before anything read them. One marks the 18 runs of six false starts, each replaced at once by a campaign of the same design. Three mark runs repeated, stopped by the stopping rule, or ended before the integrity rule read them.

Four marks leave a run counted: flagged, paused, far from its setup's other repeats, and traced by L9's or M0's recording. The stopping rule would have stopped the 14 flagged runs, taken while it only recorded. In the 222 paused runs a message arrived more than 150 ms late. Either the receiving side's disk stalled, for a cause not identified, or Kafka's broker could not reach its own controller. No run had a message arrive before it was published.

Without the flagged runs, or without all 1,110 taken while the stopping rule only recorded, no verdict moves. Without the paused runs every verdict stands but P3a on the first x86 pair, whose fitted interval sits on the edge of its ± 0.25 ms bar. It is just outside the bar with them and just inside without them. The verdict on all the runs is the one reported. The postmortem, S36, counts every mark pair by pair and says why each was given. Its four figures draw every run of the law's blocks that measured an end-to-end latency and a rate, 4,823 of their 4,838.

TABLE S23

EACH CHANGE TO THE PLAN WAS FROZEN BEFORE THE RUNS IT COVERS, AND SAYS WHETHER A RESULT PROMPTED IT. EVERY FREEZE OF THE LAW CAMPAIGN’S PLAN. PROMPTED: YES WHERE ANY OF THE FREEZE’S CHANGES FOLLOWED A RESULT ALREADY SEEN, WHICH IS NAMED. T1 TO T5 ARE THE PLAN’S TOOLS BLOCK. FREEZE 29, AN AUDIT OF PUBLISHED RESULTS, IS A SEPARATE STUDY.

Freeze	Version	Frozen	What changed	Prompted by a result?
01	5	16 Sep	The plan as registered: the law’s predictions and the rules that judge them, the blocks, the rounds rule, the integrity rule and the stopping rule	No: frozen before the first run
02	6	17 Sep	The calibration must be known within 0.3 ms; the broker’s own capture checks the added delay; Redis’s consumer acknowledges in batches of 200; M0 replays bursts; stage 0 restarts	Yes: stage 0’s first runs, which found faults in the machines, the code and two of the plan’s own checks
03	7	18 Sep	Every judgment of the added delay is read from the broker’s capture; ping is recorded and decides nothing	Yes: a sound pair turned away on a ping reading
04	8	18 Sep	A slice below a client’s zero-delay end-to-end latency is out of reach and not run, and P1 and P9 are judged on the slices a pair reaches; the stopping rule must clear its session’s own scatter	Yes: the first pair’s calibration, and a spread pilot stopped by a stopping rule set at 0.056 ms
05	9	19 Sep	P3 becomes P3a and P3b, and P3a is judged on its whole interval; L3 runs 16 rounds	No: a simulation, before any load run
06	10	20 Sep	Rounds are simulated at each broker’s own spread, never at the two pooled	No: spread pilots and a simulation
07	11	20 Sep	Each L5 session measures its own baseline at its own core count	No
08	12	20 Sep	Tools block: seven tools’ predictions fixed from their source, and one earlier classification recorded as refuted	Yes: the source audit refuted that classification; no run was read
09	13	20 Sep	P8’s program, which read an end-to-end latency L8 never runs, is corrected; L8 runs 6 rounds; the clients refuse a clock coarser than 25 μ s	No: a simulation, before any L8 run
10	14	20 Sep	P3a is judged only where the plateau reaches 2%, a lower load being out of reach; rounds are simulated at the pilots’ measured levels	Yes: L3’s first campaign, which could not place the fall at 50% load
11	15	21 Sep	Tools block: T2’s clock offsets follow each session’s own end-to-end latency, and its verdict names the behavior that survives	No: a synthetic run of the block
12	16	21 Sep	The stopping rule compares a run only with a baseline whose acknowledgment was timestamped the same way	Yes: it stopped L8 on the Arm pair at its first Python run
13	17	21 Sep	The stopping rule holds a run against its campaign’s earlier runs of the same setup, not against the calibration	Yes: it stopped L8 on the Arm pair at 28 of 144 runs
14	18	21 Sep	The stopping rule’s scatter is pooled over the campaign’s setups, and waits for five degrees of freedom	Yes: it stopped L2’s first session at 28 of 40 runs
15	19	22 Sep	P8’s program tested two of its three clauses; the priority cut is added	Yes: found while judging L8’s Arm campaign, whose verdict did not move
16	20	22 Sep	The calibration’s precision bound follows the tick; the stopping rule’s floor rises from 0.10 to 0.25 ms	Yes: three calibrations refused, and the stopping rule stopped L8 on x86 at 86 of 144 runs
17	21	22 Sep	Predictions are judged only on the runs the integrity rule passed; L2’s Kafka half moves to the first x86 pair	Yes: counting L8’s x86 runs found thirteen at a point designed for twelve
18	22	22 Sep	The calibration’s check on the publish latency is read as a slope across all its steps	Yes: a sound calibration ended on a +0.342 ms shift at 1 ms added
19	23	22 Sep	Tools block: each T2 session adds a run with no clock offset, and one tool’s two-clock setup is checked first	Yes: the first tool runs
20	24	24 Sep	L3’s Kafka campaign is added on the Arm pair, its Redis half left out, and P3b recorded as underpowered in advance	Yes, for leaving Redis out: the Arm pair’s spread pilot
21	25	24 Sep	Tools block: T2 is judged against its own control, and three tools that measured nothing run again; L3 on Arm is read at its measured loads	Yes: the check of 24 Sep
22	26	25 Sep	L5 at 2 CPUs finishes its last 13 runs with the stopping rule only recording; T2’s count check is corrected	Yes: the stopping rule stopped L5 at 2 CPUs twice
23	27	25 Sep	A 3.75 ms slice for L1, second stock-kernel days for L2 and a third x86 L3 campaign; five faults of the judging programs recorded	Yes: the check of all the data, 25 Sep, including P3a read as frozen
24	28	25 Sep	L9 is added, with the program that reads it	Yes: the same check, where the fitted shape failed
25	29	25 Sep	M0’s recording is built, and M0 runs	Yes: the same check found M0 had never run
26	30	25 Sep	The five faults of the judging programs are corrected, P6’s missing program among them; L9-2b is added before any L9 run	Yes: the same check (L9-2b: no)
27	31	25 Sep	How M0 is read, fixed before any M0 run is read	No
28	32	26 Sep	The stopping rule only records, in every campaign; stopped campaigns are finished on their own queue, calibration and kernel	Yes: five stops under the stopping rule on 25 and 26 Sep

TABLE S24

ONE PREDICTION CONFIRMED ON EVERY CAMPAIGN, THREE ON ONE BROKER, AND THE REST NOT. EVERY PRE-REGISTERED PREDICTION OF THE LAW CAMPAIGN, THE NUMBER IT WAS JUDGED ON (ALL RUNS; A SLOPE, RATIO OR MOVE WITH ITS 95% INTERVAL, FROM RESAMPLING WHOLE ROUNDS), AND ITS VERDICT, BROKER BY BROKER WHERE THE TWO DIFFER. A PREDICTION JUDGED BROKER BY BROKER IS CONFIRMED ONLY WHERE IT HOLDS ON BOTH. THE LAST COLUMN IS THE ONE VERDICT THAT MOVES WHEN THE FLAGGED, RECORDED OR PAUSED RUNS ARE LEFT OUT.

	Block, pair	The prediction	On all runs	Verdict	Moves
P1	L1, first x86	the fall's halfway point inside $[s, s + h]$ at all but one of the slices a pair reaches, at least four of them; its slope on s in 0.8–1.2	slope, free and fitted: Kafka 0.758 (0.596–1.023) and 0.230; Redis 1.007 (0.844–1.169) and 0.662	not confirmed	–
P9	L4, Arm	on the Arm pair, inside at every slice it reaches, at least two; the same slope	Kafka 0.830 (0.627–1.172) and 0.285; Redis 1.153 (0.753–1.325) and 0.798	not confirmed	–
P2	L2	the fall 2.5 to 5.5 times wider at a 4 ms tick than at 1 ms	ratio, free and fitted: Kafka 1.784 and 1.818; Redis 1.064 and 1.126 (S6.5)	not confirmed	–
P2b	L2	6 to 14 times wider at a 10 ms tick	Kafka 4.786 and 5.358; Redis 3.045 and 3.339 (S6.5)	not confirmed	–
P2c	L2	the width on the tick a line through zero, slope 0.8–1.2	fitted slope: Kafka 0.968 (0.271–1.125); Redis 0.874 (0.807–1.034)	confirmed on Redis only	–
P2d	L2	the fitted start within 0.25 ms of the slice at every tick	within it at one of twelve slice and tick pairs	not confirmed	–
P3a	L3, five Kafka campaigns	the halfway point moves less than 0.25 ms from 75% to 88% load	free move: $-0.009, -0.188, -0.086, -0.011$ and -0.309 ms	not confirmed on any	first x86, 19 Sep
P3b	L3, five Kafka campaigns	load raises the plateau	rise, free: 0.048 (0.044–0.052), 0.047, 0.048, 0.041 and 0.050	confirmed on all five	–
P4	L7, first x86	real-time priority cuts the plateau at least five times wherever it is at least 2%	Kafka 0.0405 to zero; Redis's plateau 0.0069 (S6.5)	confirmed on Kafka only	–
P6	L6	a recording of every thread's waits predicts the rate within 0.67–1.5	median ratio 2.49 to 13.08 over four pair and broker parts; no setup inside	not confirmed	–
P7	L5, second x86	the default slice follows the core count, and the fall follows it	slices read back at 2, 4 and 8 CPUs as the rule gives; the fall on Kafka at 8 CPUs, Redis's fitted shape missing at all three	confirmed on Kafka only	–
P8	L8, first x86 and Arm	Java has the same fall and priority effect, and Python's plateau is at most 1.5 times Java's	Python over Java: 3.251 (2.795–3.907), and 4.253 (4.013–4.560) on Arm	not confirmed	–
L9-1	L9, three pairs	at least 90% of negative readings mostly spent waiting for a CPU	79.4 to 92.0%	not confirmed	–
L9-2	L9	the time-weighted waits predict the rate within 0.67–1.5	median ratio 0.104 to 0.165	not confirmed	–
L9-2b	L9	the share of acknowledgments whose own wait outlasted $D - g$ predicts it	inside the rule on four of six parts; median ratio 1.11 to 1.30	not confirmed	–
M0	M0, first x86	four named causes of the end-to-end latency's slope under bursts	slope on the added delay, traced half: Kafka 0.995; Redis 1.181 and 1.307; every cause dropped for Redis, two not tested on Kafka	–	–

S6.5. Read after the verdicts: the wait is the rest of a slice, and two judges departed from the plan

Everything here is post hoc. It was read after the verdicts, and none of it changes one. The rate holds no plateau. At half the slice it is 1.45 to 3.94 times its value at 0.9 of the slice, in each of the ten slice and broker combinations whose runs reach half the slice.

L9's recordings, at a 3 ms slice and a 1 ms tick, time the wait of the thread that timestamps each acknowledgment, for 1,516,342 acknowledgments in 304 runs. Of those whose thread waited more than 1 ms, 24.3 to 35.8% were still waiting at 3 ms and 3.3 to 6.8% past 4 ms, over twelve pair, broker and load parts. A thread preempted for a task starting its slice waits out all of it, but a thread woken while another task holds the core waits out what is left. The acknowledgment's reply wakes the thread that timestamps it, so its wait is the second kind. A whole-slice wait would leave all of them still waiting at 3 ms. A wait for the rest of the slice leaves 20% waiting at 3 ms and none past 4 ms. That thread waits out what is left of the running task's slice, R , uniform on $[0, s]$, and then up to a tick, U , uniform on $[0, h]$, so that for $h \leq s$

$$G(x) = \Pr[R + U > x] = \begin{cases} 1 - x^2/(2sh), & 0 \leq x \leq h, \\ 1 - (x - h/2)/s, & h \leq x \leq s, \\ (s + h - x)^2/(2sh), & s \leq x \leq s + h, \\ 0, & x \geq s + h. \end{cases} \quad (\text{S5})$$

Fitted to a rate that falls from the shortest end-to-end latency on, the law's shape puts its fall below the slice and moves it less than the slice moves. That explains the split between the free and fitted readings of P1 and P9 (postmortem, S16.10). The

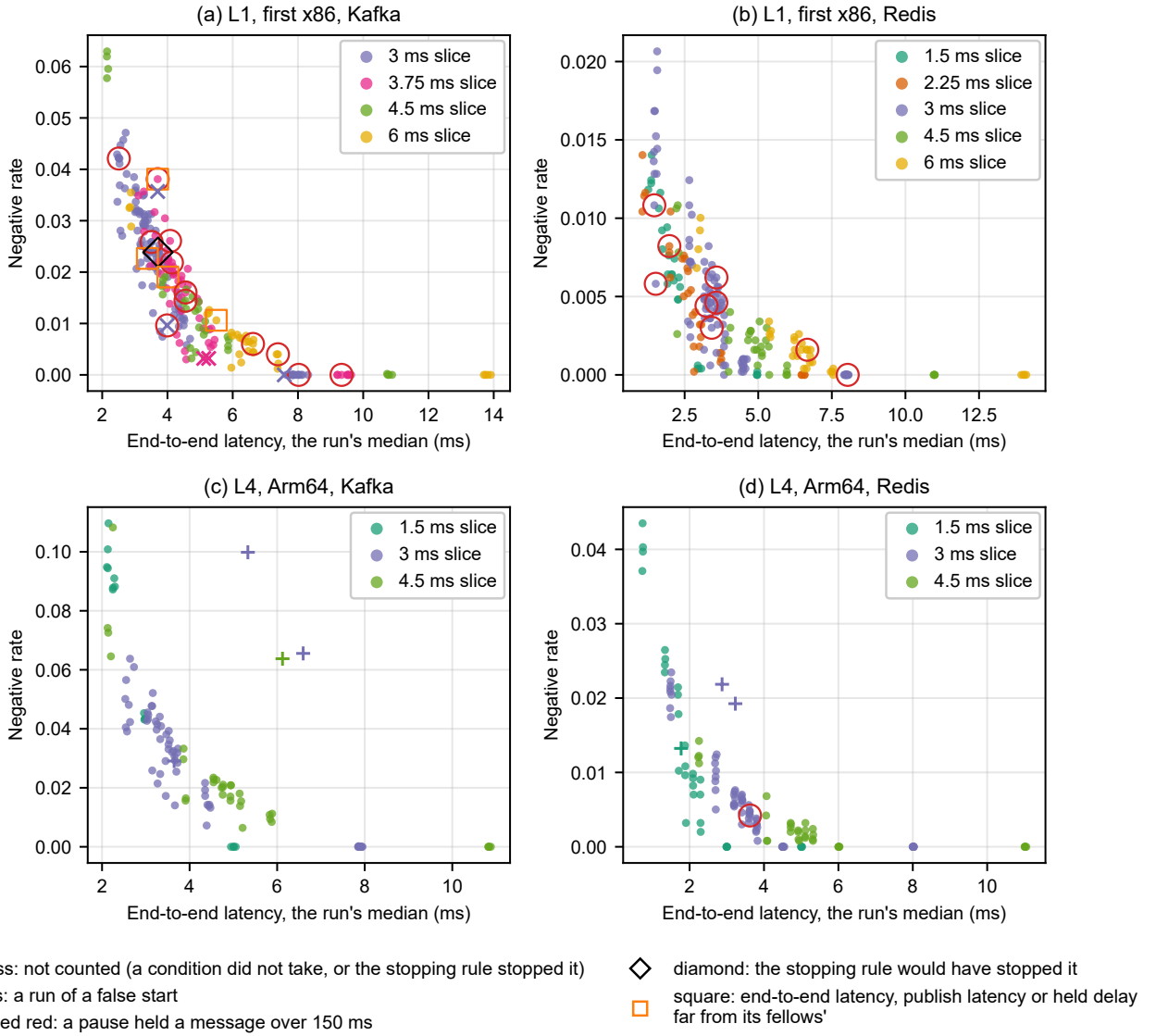


Fig. S10. **Slice runs fall from the shortest end-to-end latency on, where the registered law put a plateau.** Every run of L1 (first x86 pair) and L4 (Arm pair, Arm64 in the panels) at its median end-to-end latency and its negative-span rate, colored by the slice it ran at. Crosses: runs the integrity rule did not count. Plusses: runs of L4's two false starts. Rings, diamonds and squares: counted runs with a warning (S6.4). That the rate holds no plateau was read after the verdicts (S6.5).

recording raised the rate it recorded, 1.36 to 1.49 times for Kafka and 3.69 to 4.33 times for Redis, so these waits describe the machines with it on.

In two places a frozen program departed from the plan, and in both the frozen answer is the one reported. The programs judging P2, P2b and P2c group L2's runs by tick alone, so each width they read spans the falls of both its slices, 1.5 and 3 ms. With Equation S4 in place of every run's rate, as if the law held exactly, the P2 program returns 1.98 free and 1.57 fitted for Redis, below its band. One slice at a time, the same formula gives 4.0 to 5.2 for P2 and 9.8 to 12.0 for P2b, inside both bands. Those three verdicts therefore carry no evidence about the law either way. P4's program words Redis "not confirmed" although its plateau, 0.69%, lies below the 2% at which P4 makes a claim. The plan's rule reports a plateau that low as out of reach. What explains M0's slope on Redis was also found afterwards (postmortem, S37).

S7. THE BROKER COMPARISON, GATED

Once the runs the sign check condemns are removed, Kafka and Redis are equivalent on the event-time latency. Which is faster depends on a consumer setting, and a co-located testbed, with broker and clients on one host, hides its effect.

The comparison is stated first on the event-time latency E of Equation 2, because E is a causal chain, each step preceding the next, and the transport proxy S is not (Section IV-A). Over the runs the sign check leaves standing, E is equivalent between the two brokers against a 40 ms margin, one broadcast frame at 25 frames per second, fixed three days before the campaign

ran. The Hodges–Lehmann shifts are 0.682–0.859 ms, Kafka slower, and the highest upper bound of any 90% interval of the three estimators is 0.906 ms. E runs to t_{out} , so it also contains both clients’ payload parse, which S and D carry for Kafka alone (Section VI-D). On S the two sit within a millisecond. At one feed the Hodges–Lehmann shift is 0.408 ms (90% bootstrap interval 0.389–0.419), and two one-sided tests (TOST) reject both nulls, a shift beyond either edge of a 1 ms margin, at $p < 0.001$. The shift holds between 0.408–0.417 ms across three concurrency levels, and Welch’s test, a percentile bootstrap and the Hodges–Lehmann estimator agree on equivalence at every level.

The two brokers are equivalent within a margin, yet still distinguishable. The numbers on S come from a powered campaign with 15 replicates at each of the three concurrency levels and a median of 125 events per run. It replays at a rate derived from the plan and verified against wall time. It timestamps in microseconds on one clock, so Failure 2, which needs millisecond timestamps, cannot touch its numbers. Failure 1 can, and the sign check removed the runs it condemned (S2.2). On S , Kafka sits near 0.54 ms and Redis near 0.11 ms, and the 0.41 ms shift, Kafka slower, sits far from zero at every level, in the direction the gray-literature comparisons report.

A second powered campaign, run ten hours later with the same protocol, reproduces the shift and fails the criterion registered for it, that its shift fall inside the first campaign’s 90% interval at every level. Over the runs the check keeps, its three shifts are 0.381, 0.414 and 0.416 ms. The first, from the two Redis runs kept at one feed, falls 0.008 ms below the interval. We saw only after the result that the criterion was a poor test. A second estimate carries its own sampling error, so even with identical true shifts it would be met at all three levels only about 38% of the time. Restoring the runs the sign check condemned moves the shift by at most 0.016 ms (S2.2).

Part of even this shift comes from our harness. We timestamped Kafka’s acknowledgment on the publishing thread, as Redis’s already is, with one request outstanding under matched load. That shrank the gap by 0.071 ms (95% bootstrap over runs: 0.039–0.113), and by 0.039 ms (95% bootstrap over runs: 0.008–0.059) in a replication at 30 runs per cell. What remains is a broker difference of 0.34–0.37 ms. The bias never turns a value negative, so the sign check, which rejects 40 of the 160 runs behind these numbers, cannot see it. Sizing a bias of a few hundredths of a millisecond took a controlled experiment and a replication.

A consumer that acknowledges each message and waits for the reply reverses the order, and injected delay exposes it (Table S25). With one-way delay injected identically at both brokers, Redis’s median event-time latency reaches 4,651 ms at 5 ms of delay and 103,143 ms at 50 ms, against Kafka’s 12.4 and 336.6 ms. Redis’s transport proxy grows about 930 to 2,050 times the delay applied, while Kafka’s grows by about the delay itself. The cause is the consumer pattern, not the broker. Our Redis consumer acknowledges each entry and waits for the reply, the pattern the tutorials teach, so a read of two hundred entries is followed by two hundred sequential round trips. Each trip costs almost nothing at 0.22 ms. At 20 ms the consumer serves entries more slowly than they arrive, so a backlog builds and sets the delay. Kafka’s consumer serves from a prefetched buffer and commits on a timer, so it pays the delay once per batch.

TABLE S25

PER-MESSAGE ACKNOWLEDGMENT TURNS INJECTED DELAY INTO SECONDS OF BACKLOG FOR REDIS ALONE. MEDIAN EVENT-TIME LATENCY E AND TRANSPORT PROXY S UNDER ONE-WAY DELAY INJECTED IDENTICALLY AT BOTH BROKERS, FIVE FEEDS PER CONFIGURATION. THE ZERO-DELAY ROW IS REJECTED IN BOTH CONFIGURATIONS, SO THERE IS NO USABLE BASELINE.

Delay	Kafka E	Kafka S	Redis E	Redis S
0 ms		rejected		
5 ms	12.4 ms	5.6 ms	4,651 ms	4,645 ms
20 ms	77.8 ms	20.8 ms	31,401 ms	31,268 ms
50 ms	336.6 ms	44.9 ms	103,143 ms	102,460 ms

The outcome that would refute this explanation was recorded before the run. If the acknowledgment pattern is the cause, batching the acknowledgments must remove most of the effect without touching the broker, the network or the injected delay. At one feed and 20 ms of delay, batching them two hundred at a time cut the median event-time latency from 4,138 ms to 103 ms, a factor of 40, replicated four times. Read-loop instrumentation shows each read returning every waiting entry, a median of 106, in about 32 ms, so the reads keep up and the bottleneck is the acknowledgment path. At five feeds batching left the backlog in place, 92.4 s batched against 87.6 s unbatched, and why batching did nothing there is unexplained (postmortem, S19).

A co-located testbed cannot see any of this. There the round trip is a few hundred microseconds, so paying it once per event or once per batch differs by less than the noise. The setting that decides the comparison under any realistic separation is therefore invisible where most comparisons are run. That is the shape of the paper’s two failures: a configuration decides the result, and nothing in the output records it.

S8. RESULTS WE WITHDREW

The sign check condemned our own first result, which had passed every conventional check we knew. A second early result, a twentyfold gap in event-time latency between the brokers, rested on Kafka runs that all passed the sign check and still measured only the harness starting up. Table S28 lists every other result we withdrew or withheld.

TABLE S26

EVERY CELL OF OUR FIRST RESULT SET FAILS THE SIGN CHECK. THE TRANSPORT PROXY S (MEDIAN, MS) BY CONCURRENCY N , WORKSTATION TESTBED, $10\times$ REPLAY, WITH THE HOLM-CORRECTED p OF EACH COMPARISON.

N	Kafka	Redis	Difference	p_{Holm}
1	0.999	1.006	0.007	1.0×10^{-4}
5	1.001	1.197	0.196	6.5×10^{-6}
10	1.002	1.346	0.344	9.0×10^{-11}

S8.1. The first result set passed every conventional check

An invalid corpus can look decisive. On the workstation testbed we replayed real matches at ten times real speed over one, five and ten feeds, with 15–30 runs per cell. The transport proxy separated the brokers cleanly: every comparison was significant after correction, and at $N \geq 5$ the run distributions did not overlap (Table S26). The sign check of Section II-C rejects 862 of the testbed’s 1,382 runs, 109 of the 126 behind the table among them. A condition is usable only if all of its runs survive, so none of the table’s 6 conditions is usable. Of the testbed’s 76 conditions, 8 survive. The table reports S , which our first harness timed from the acknowledgment, and S is negative whenever the publish latency exceeds the end-to-end latency (Equation 3).

The corpus was not naive. Five earlier corrections had shaped it, each found by investigating a result that looked wrong and each reversing the apparent winner:

- 1) A process-relative clock. Each process had its own clock zero, so each run’s consumer start-up offset entered every measurement.
- 2) A replay speed of $120\times$, at which the producer could not keep pace.
- 3) An asymmetric load generator. The Kafka producer waited a broker round trip per event, and pipelining it cut median event-time latency at one feed from 1,644 ms to 16 ms.
- 4) A concurrency axis that was covertly a throughput axis.
- 5) A heavy tail in every mean. The 95th percentile of Kafka’s event-time latency sat at 99–105 ms at every concurrency, and decomposition placed it in publish delay.

A fairness audit found payloads and serialization costs comparable. A sixth problem was an option the orchestration script accepted and never passed on, so the first run of the acknowledgment experiment never applied its treatment and showed no effect. Since then every intervention carries a manipulation check that aborts the campaign if the treatment is not accepted.

The corpus had survived six corrections, a fairness audit, and a full battery of rank tests, effect sizes and multiplicity correction. Yet none of its conditions was usable, and nothing in its output said so.

S8.2. The twentyfold gap in event-time latency was publish delay at the kick-off

A corpus can pass the sign check and still measure its own start-up. On the cloud testbed the historical comparison E1 put the brokers’ median event-time latencies near 105 and 5 ms. Of the 105.5 ms for Kafka, 102.9 ms was publish delay, while the transport proxy was 0.84 and 0.80 ms for the two brokers. Our first analysis reported the gap as a finding, attributed it to the client’s sender thread waking from idle, and built a practitioner recommendation on it. E1’s Kafka runs all passed the sign check, and its four cells’ medians agreed within half a millisecond, 105.3 to 105.7 ms.

That publish delay is paid once per run. Replaying one match at a verified real-time rate under three observation windows, all else fixed, we counted the events that woke more than 50 ms late (Table S27). As the events emitted grow from 57 to 507, that count stays at four and exactly one publish call blocks. The share paying the cost therefore falls from 7.0% to 0.8%, as $1/\text{events}$, which is the dilution a once-per-run cost predicts. A per-event cost would have grown the count with the window. The median publish delay of Kafka stays at 1.6 ms throughout, and its maximum at 103.5 ms.

The per-event trace shows why. The first publish call blocks for 102.6 ms, and the single-threaded replay loop wakes each event that fell due meanwhile about 103 ms late. A Kafka producer fetches metadata before its first publish to a topic it has not yet cached [S26]. The block fits that fetch, although we did not isolate the call. Every plan opens with two lineup records, two half-start markers and the kick-off pass, all at second zero, so in the plan behind the sweep the first five events share one timestamp. The first pays the block and the other four wait it out. A football feed therefore delivers its densest burst when the producer is coldest. The Redis producer has no such prologue, because `XADD` to a new stream creates it in the same round trip.

E1 matched a median of seven events per run. A median of seven values at 102.93 ms needs at least four of them that late, and the trace shows exactly four per run, so the matched sample consisted almost entirely of events the prologue delayed. The argument holds at every replay rate E1 could have used (postmortem, S3). The twentyfold figure is withdrawn, and the broker comparison now rests on powered campaigns that match a median of 125 events per run (Section VI-D). A percentile computed over fewer than ten events describes the harness.

TABLE S27

COUNT OF LATE EVENTS DOES NOT GROW WITH THE WINDOW. SAME MATCH, DRIVER AND BROKER, ONE FEED, TRUE REAL-TIME REPLAY; ONLY THE OBSERVATION WINDOW CHANGES. BOTH PRODUCERS ARE LOOP-TRACED. COUNTS ARE MEDIANS OVER THREE REPLICATES FROM THE PER-EVENT TRACE; THE PUBLISH-DELAY PERCENTILES COME FROM THE PER-RUN SUMMARY.

	Window (s)	Events		Publish delay (ms)		Events > 50 ms late	Blocking publishes
		emitted	matched	median	max		
Kafka	60	57	57	1.56	103.4	4	1
	180	148	148	1.61	103.5	4	1
	600	507	465	1.58	103.5	4	1
Redis	60	57	57	1.50	17.4	0	0
	180	148	148	1.56	20.4	0	0
	600	507	465	1.53	24.8	0	0

TABLE S28

EVERY OTHER RESULT WE WITHDREW OR WITHHELD, AND WHAT DEFEATED IT. ONE ROW PER RESULT, WITH THE SECTIONS OF THE POSTMORTEM THAT TELL IT IN FULL.

What we claimed	Why it fell	Postmortem
The benchmark's discards were causality violations.	Separated by sign, the Kafka-driver corpus's 10,913,263 discards hold not one negative. They are resolution failures.	S1.3
A candidate mechanism for the Redis-driver negatives: skew between two views of a virtualized clock.	The driver takes the publish time from the entry identifier the Redis server assigns. Its span is a difference between two millisecond-floored clocks on two hosts, for which $-1000\mu\text{s}$ is the only negative value expressible.	S1.3
Failure 1 was a causality violation.	The acknowledgment and the receipt are two branches of one cause. Recounted, S is negative on 62,264 of 738,730 events and the publish-timed span on 0.	S1.4
An M/G/1 waiting time, then our own registered bracket, described how the rate grows with utilization.	Over $\rho = 0.881\text{--}0.990$ M/G/1 fitted worse than the mean ($R^2 = -0.05$), and the bracket predicted $2.45\text{--}3.07\times$ growth against $1.44\times$ measured. Real-time priority showed that ρ is not the variable.	S1.5, S5.1
A traced tail index confirmed the payload sweep's exponent.	Two proper estimators on the same histogram disagree sixfold (0.21 and 1.19). The agreement was a coincidence of window and estimator.	S1.6, S8
Six predictions registered before the comprehensive campaign, and a jitter-kernel conjecture after it.	One prediction was confirmed, three falsifiers fired, one became unevaluable and one split. An alternation test killed the conjecture.	S1.7, S1.15
The achieved publish rate, for part of the campaign.	It was copied from the configuration flag, which for a subset of runs disagreed with the rate against elapsed wall time. The affected conditions were re-derived from per-event timestamps.	S1.8
A 2026 comparison gave Redis a p99 of 0.8 ms against Kafka's 12.5 ms.	Those figures are not on the page cited. They belong to another comparison that a search returned beside it; the postmortem cites both.	S1.9
The benchmark's cross-host exposure, measured by running it.	Its distributed mode failed on every attempt, across two releases. The exposure rests on its source and our own two-host harness.	S1.10, S22.1
The injected-delay sweep (H1): a rank correlation across 0–50 ms of delay at the broker supported the rate's fall as the end-to-end latency lengthens.	In the re-run at one feed the delay was common-mode and canceled in S : over 0–20 ms event-time latency tracked it, 3.72 to 23.61 ms, while S stayed flat.	S3.1
A recorded replay rate for every run we report.	Plans carry a built-in $120\times$ compression, so the speed-up flag is not the rate, and for the earliest corpora, E1 among them, no surviving artifact records the achieved one. For E1 the start-up cost reads the rate back as true real time, and the original script, found later, agrees.	S4
A co-location campaign, meant to shorten the end-to-end latency.	Moving the broker onto the driver lengthened S at idle, 0.512 to 0.573 ms, and moved it $1.03\times$ under load. Nothing was manipulated, so nothing was tested.	S4.1
The two brokers were equal on the transport proxy, from E1.	E1 matched seven events per run. The powered runs' first seven events reproduce its near-equality; over all events Redis is 0.41 ms faster.	S2, S5
Configurations whose publish interval over the timestamp resolution has an even denominator ($q = 2, 4$) could not test the grid.	An on-grid configuration is predicted flat, so they are evidence for the phase account. The exclusion also flipped under a rate-local estimate of the continuous value.	S6
At $q = 5$, retention near 40% (registered).	The median was 51.04%. The replicates split about 40% and 60%, the two branches the phase account requires.	S6

S9. RELATED WORK IN FULL

Clock resolution and scheduling delay are each well understood, and rules of the form we propose already exist in neighboring fields. What the literature lacks is their consequence for the sub-millisecond comparisons that broker benchmarks now report, whose tools mostly discard the evidence that would show it.

S9.1. Benchmark variability is structural, and here it has a mechanism

Run-to-run instability in benchmarks is structural, and the literature that shows it supports our reading. Mytkowicz et al. showed that factors as innocuous as link order and environment-variable size flip published conclusions [S27], and Georges et al. that JVM results without confidence intervals routinely mislead [S28]. Barrett et al. found that language virtual machines, such as the JVM, frequently never reach the steady state their benchmarks assume, so the distribution being sampled is itself non-stationary [S29]. That is the closest published analogue of the mobility we see between identical runs. Maricq et al. found hundreds of trials necessary for stable comparisons [S30].

Only common practice assumes otherwise. The norm that surveys report, a handful of repetitions, presumes a stable distribution [S31], [S32]. What we add is the mechanism. Where the publish interval is commensurate with the timestamp resolution, in a ratio of small whole numbers, the instability is not variance around a value. It is a distribution on two values whose weights move between runs, so no replicate count converges (Section V-E).

Tail latency under load is established, and what is new is the measurement it corrupts. Li et al. decompose the tail latency of three Linux servers and find measured tails substantially worse than a queueing model alone predicts [S33]. The excess comes from background processes, request re-ordering, interrupt routing, power management and NUMA effects. We add that the stall distribution also enters the measurement of an unrelated quantity, and that load grows its tail far faster than its narrow central part. From an idle host to the heaviest load before its processors are fully busy, the negative-span rate grows $61\times$ and the jitter's width $5\times$ (S3.4). A benchmark can therefore sit at a utilization where its measurements look tight, because the jitter is narrow, while the tail that corrupts them is already growing.

S9.2. The resolution literature knew the problem, and the deletion is new

The problem behind Failure 2 is older than every tool Section V-A reads. Danzig and Melvin built microsecond timers for workstations in 1990 because the available clock was too coarse for the interval being timed [S34]. The arithmetic is older still. Hewlett-Packard's Application Note 162-1 already contains the retention identity, the reduced fraction that sets the lattice, the ceiling on resolution gain, the symptom and the cure [S6]. The one thing it does not contain is the failure this paper reports, because its counter keeps both readings and uses the retained fraction as its estimator. A positivity filter deletes the readings that carry the fractional part, and so turns a solved metrology problem back into an open one.

The mathematics is a century old. Weyl's equidistribution theorem [S35] governs the long run when the publish interval and the timestamp resolution are incommensurate, and the three-distance theorem [S36] governs the phases a finite run of publishes visits. Schuchman's condition [S37], generalized by Wannamaker et al. [S38], underlies the dithering that Table III recommends. The timing literature treats a coarse timestamp as staleness, not deletion.

Coordinated omission is the closest neighbor, and its mirror image [S39]. There the worst samples are never taken, so percentiles are optimistic by orders of magnitude. Under Failure 2 the samples are taken and the latency is computed, and a downstream filter rejects it. One loses the slow tail, the other the fast bulk, and a correction for one does nothing about the other. The benchmark we instrument records into an HdrHistogram, which ships coordinated-omission machinery and none for this failure [S40]. Its sound rejection of negative values is what motivated the positivity filter, so a protection against one measurement error created the conditions for another.

What we claim as new is narrow. Quantization is elementary, and Cloudprofiler exists because milliseconds are known to be too coarse [S41]. The new element is the interaction of a quantized timestamp, a producer with few publish phases and a positivity filter. It yields a retained fraction that is bimodal, irreproducible between identical runs and invisible in the reported summary. Its damage is set by the denominator, in lowest terms, of the publish interval over the timestamp resolution.

S9.3. Rules of the right form exist, in other fields

Standardized benchmarking already contains rules of the form we propose. CoreMark-PRO requires each workload to run for at least 1000 times the minimum timer resolution [S42]. That is a floor on the measured interval relative to the timestamp resolution, the ratio Equation 6 turns on. Network measurement goes further and keeps the impossible value as data (Table S29).

The flagship standardized benchmark for message-oriented middleware has none of this. SPECjms2007 records end-to-end latency in a histogram of 100 ms default granularity [S48], [S49]. Its run rules, design document and user's guide contain no resolution clause, no minimum measurable interval and no counter of discarded latency samples. Their only timing-quality requirement, clause 3.2, bounds inter-node clock agreement at 100 ms, which is a skew limit. The guide does discard throughput samples, in numbers fixed by its configuration, but it never counts the latency samples its own resolution makes unusable.

The nearest prior sighting of a negative span inside a benchmark was blamed on a clock. Klenik and Pataricza report negative derived durations in a Hyperledger Fabric benchmark, call them a serious violation of event causality, and trace them to a drifting peer clock [S50]. The durations oscillate smoothly around zero, the signature of a drifting clock, which puts them with our Redis-driver negatives of $-1000\ \mu\text{s}$, also from two clocks, and outside Failure 1. Their check is post-hoc data validation, with no per-run gate and no campaign rate.

TABLE S29
RULES OF THE RIGHT FORM ALREADY EXIST, IN NEIGHBORING FIELDS. EACH ADDRESSES THE TIMESTAMP RESOLUTION OR THE SAMPLES A MEASUREMENT CANNOT USE; NONE IS APPLIED WHERE A BROKER BENCHMARK REPORTS A LATENCY.

Source	What it requires or does	How far it reaches
CoreMark-PRO run rules [S42]	Each workload runs for at least 1000 times the minimum timer resolution.	A floor relative to the resolution. Plain CoreMark sets an absolute 10 s floor instead.
SPEC power methodology, clause 8.2.2 [S43]	Should report the samples that may return “Unknown” and those exceeding the maximum allowed uncertainty.	Counts unusable readings; a “should”, in a methodology document.
OWAMP [S44]	Records a packet timestamped in the future unaltered, within its timeout window.	Keeps an impossible value as data.
ETSI TS 102 250-4 [S45, Clause 5.1]	Requires removed measurement cycles to be reported.	Counts what was removed.
RFC 7679 [S46, Sec. 3.5]	Treats negative singletons as usable distribution data.	Keeps the sign.
TimeWeaver [S47]	Keeps measurements with missing timestamps or negative latencies, in lower accuracy tiers. States that the prior method rejected about 3,631 of 8,804 clients.	Retention with a quality label, and a reported rejection rate. Network measurement, counting clients.

S9.4. The run queue inverts spans that a backlog cannot

A span can invert with no backlog at all, and that is where our reading parts from the concurrent report. Sharma et al. count negative timing spans in distributed inference systems and propose a `causality_health` check [S51]. Injecting skew, they observe no violations up to 3 ms and clear ones by 5 ms, and read that as a skew threshold. We do not dispute their measurements, only one sentence of the reading: “queueing alone cannot produce negative timing spans”.

Under the queue they model, a backlog at a service stage, the sentence is true, and Section IV also rules out a backlog as the cause of our own negatives. As written it is false, because two queues are in play. A message waits in a service backlog. The thread that reads the clock waits in the kernel’s run queue for a core. The second wait delays a timestamp whether or not any backlog exists, and our rates are highest at high utilization with no backlog at all.

If a backlog cannot invert a span, removing the backlog leaves skew as the only explanation, and a millisecond threshold on skew becomes a sufficient check. On a loaded machine it is not. On a single host, where one clock leaves no inter-host offset at all, we see a negative-span rate of 30.5% at 88% utilization, and a scheduling parameter moves it fiftyfold without touching the clock (Table II). Our cross-host clocks cannot settle the question either way. Between our hosts `chrony` reports an offset of 67 μ s, far inside the 3 ms at which Sharma et al. saw no violations, but it bounds its own error at 4–7 ms per host. We therefore claim no bound on the offset between hosts (Section VIII), and the single-host rate needs none.

The skew reading is older than this dispute. The benchmark’s maintainer gave it in 2021, attributing publish latency above end-to-end latency to inter-node skew, although the report came from a single node [S52]. It does not consider whether the thread that took the timestamp was running.

Their own formulation is nearer ours than the split between queueing and skew allows. They note that the minimum separation between the two events “exhibits variability due to scheduling jitter”, and that violations grow likelier as the skew exceeds the quantiles of that separation. That is the condition of Equation 3 with skew written where we write the stall. A threshold in milliseconds of skew cannot transfer to a machine where the stall is the term that grows.

S9.5. Production tracing reached the conclusion without the mechanism

Production tracing concluded, from the symptom alone, that rewriting an inverted span’s timestamps does harm. Distributed tracing systems ship clock-skew adjusters that “modif[y] start time and log timestamps for spans that appear to be ‘off’ with respect to the parent span” [S53]. An issue titled “Clock Skew Adjuster considered harmful” called the result “wrong data and really hard to diagnose” [S54], and Jaeger has defaulted the adjustment off since 2020. A July 2026 bug against the OpenTelemetry eBPF instrumentation reports child spans starting before their parents on same-node resources, from a reporter who had checked for clock skew and found none [S55]. Its deltas of seconds to minutes are orders of magnitude beyond any run-queue stall, so their cause is not Failure 1. The report shows the absence of a second hypothesis: a practitioner who has ruled out the clock, on one node, has nowhere else to look. If the negative span comes from a late timestamp, an adjuster moves evidence instead of correcting an error, by an amount Section IV measures.

S9.6. A runnable thread does not always run

Work-conservation violations leave runnable threads waiting while cores sit idle. Lozi et al. document Linux scheduler bugs that do so for seconds, invisible to conventional testing because throughput stays healthy [S56]. Those are CFS-era defects. Our kernel’s scheduler is EEVDF, and we make no claim that they are present.

Queueing gives the leading-order account of waiting, and it is right about what matters here. Utilization alone does not fix a waiting time, because at fixed ρ the pooling of servers and the load geometry matter. The geometry contrast of Section IV-B therefore refutes a single-parameter law and leaves standing multi-server queueing, which is geometry-dependent at fixed ρ .

Chandrasekar and Kramberger apply such a single-parameter M/G/1 framing to benchmark bias [S57]. We adopted it too, and on the load axis it fitted worse than the mean (Table S28). On a second axis they reach half of our mechanism, concurrently and independently: their client’s own interpreter lock “artificially inflates” the latencies it reports, charging the harness’s scheduling to the server under test. That is the inflation half. What a positive number cannot show remains ours: the negative span, and the sign channel that shows the acknowledgment was timestamped late.

Li et al. show real-time priority collapsing a tail that renicing leaves alone, on the system under test [S33]. We apply the same lever to the harness (Section IV-B). An agent acting on a lagged observation is a problem of partial observability, studied in this journal for load balancing under delayed feedback [S58]. There the lag is a known property of the system. Here it sits inside the benchmark, unknown to those reading its output.

REFERENCES

- [S1] StatsBomb, “Statsbomb open data,” *GitHub Repository*, 2023. [Online]. Available: <https://github.com/statsbomb/open-data>
- [S2] M. Mohammad, “Analysis of design patterns and benchmark practices in Apache Kafka event-streaming systems,” *arXiv preprint arXiv:2512.16146*, 2025.
- [S3] Linux Kernel Documentation, “KVM-specific MSRs: MSR_KVM_SYSTEM_TIME_NEW,” <https://docs.kernel.org/virt/kvm/x86/msr.html>, 2026, accessed: Aug. 21, 2026.
- [S4] Z. Zhou, “sched: Reduce the default slice to avoid tasks getting an extra tick,” Linux kernel mailing list, commit 2ae891b82695, 2025.
- [S5] S. Bradner, “Benchmarking terminology for network interconnection devices,” Internet Engineering Task Force, RFC 1242, 1991.
- [S6] Hewlett-Packard, “Time interval averaging,” Hewlett-Packard Company, Application Note 162-1, 1970, part no. 02-5952-0766.
- [S7] J. H. Gliever and D. C. Chu, “Measurement averaging counting apparatus employing a randomly phase modulated time base to improve counting resolution,” U.S. Patent 3,938,042, Feb., 1976, assigned to the Hewlett-Packard Company. Accessed: Sep. 11, 2026.
- [S8] IEEE, “IEEE standard for terminology and test methods for analog-to-digital converters,” IEEE Std 1241-2010, 2011.
- [S9] Joint Committee for Guides in Metrology, “Evaluation of measurement data — guide to the expression of uncertainty in measurement (JCGM 100:2008),” https://www.bipm.org/documents/20126/2071204/JCGM_100_2008_E.pdf, 2008, accessed: Aug. 20, 2026.
- [S10] J. W. Pratt, “Remarks on zeros and ties in the Wilcoxon signed rank procedures,” *Journal of the American Statistical Association*, vol. 54, no. 287, pp. 655–667, 1959.
- [S11] Apache Software Foundation, “Apache Kafka tools — EndToEndLatency.java and ProducerPerformance.java,” Source repository, 2026, <https://github.com/apache/kafka>. Accessed: Aug. 31, 2026.
- [S12] Apache Pulsar contributors, “PerformanceConsumer.java,” Source repository, 2026, <https://github.com/apache/pulsar>. Accessed: Aug. 21, 2026.
- [S13] Synadia, “NATS cli — cli/latency_command.go,” Source repository, 2026, <https://github.com/nats-io/natscli>. Accessed: Oct. 2, 2026.
- [S14] OpenMessaging Project, Linux Foundation, “The OpenMessaging benchmark framework,” 2018, available at openmessaging.cloud/docs/benchmarks/. Accessed: Aug. 19, 2026.
- [S15] Broadcom, “RabbitMQ PerfTest — TimestampProvider.java,” Source repository, 2026, <https://github.com/rabbitmq/rabbitmq-perf-test>. Accessed: Oct. 2, 2026.
- [S16] IOP Systems, “Rezulus: systems performance telemetry — scheduler_discarded_samples,” Source repository, 2026, <https://github.com/iopsystems/rezulus>. Accessed: Aug. 20, 2026.
- [S17] A. D. Brunelle, “blktrace/btt — btt/inlines.h,” Source repository, 2026, accessed: Aug. 20, 2026.
- [S18] EMQ Technologies, “emqtt-bench: MQTT benchmark tool — src/emqtt_bench.erl,” Source repository, 2026, <https://github.com/emqx/emqtt-bench>. Accessed: Aug. 21, 2026.
- [S19] J. Axboe, “fio: Flexible I/O tester — gettime.c,” Source repository, 2026, <https://github.com/axboe/fio>. Accessed: Aug. 20, 2026.
- [S20] G. Tene, “wrk2 — src/wrk.c,” Source repository, 2026, <https://github.com/giltene/wrk2>. Accessed: Aug. 21, 2026.
- [S21] Apache Kafka contributors, “KAFKA-19888: Clamp negative values in coordinator histograms,” Issue tracker and pull request #20986, 2025, <https://issues.apache.org/jira/browse/KAFKA-19888>. Accessed: Aug. 21, 2026.
- [S22] —, “Kafka Streams StreamTask.java: end-to-end latency recording (KIP-613),” Source repository, 2026, <https://github.com/apache/kafka/blob/trunk/streams/src/main/java/org/apache/kafka/streams/processor/StreamTask.java>. Accessed: Sep. 13, 2026.
- [S23] S. Guo, “Avoid adding negative metric values (pull request #56),” OpenMessaging Benchmark, commit ebb5d6b8, Mar. 2018, <https://github.com/openmessaging/benchmark/pull/56>. Accessed: Aug. 18, 2026.
- [S24] Confluent, “librdkafka introduction — delivery reports,” <https://github.com/confluentinc/librdkafka>, 2026, accessed: Aug. 20, 2026.
- [S25] J. Axboe, “fio HOWTO — interpreting the output,” <https://fio.readthedocs.io/>, 2026, accessed: Aug. 20, 2026.
- [S26] Apache Kafka, “KIP-526: Reduce producer metadata lookups for large number of topics,” Apache Software Foundation, Kafka Improvement Proposal, 2020.
- [S27] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, “Producing wrong data without doing anything obviously wrong!” in *Proc. 14th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 2009, pp. 265–276.
- [S28] A. Georges, D. Buytaert, and L. Eeckhout, “Statistically rigorous Java performance evaluation,” in *Proc. 22nd ACM SIGPLAN Conf. Object-Oriented Programming, Systems, Languages and Applications (OOPSLA)*. ACM, 2007, pp. 57–76.
- [S29] E. Barrett, C. F. Bolz-Tereick, R. Killick, S. Mount, and L. Tratt, “Virtual machine warmup blows hot and cold,” *Proceedings of the ACM on Programming Languages*, vol. 1, no. OOPSLA, pp. 52:1–52:27, 2017.
- [S30] A. Maricq, D. Duplyakin, I. Jimenez, C. Maltzahn, R. Stutsman, and R. Ricci, “Taming performance variability,” in *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 2018, pp. 409–425.
- [S31] E. van der Kouwe, G. Heiser, D. Andriesse, H. Bos, and C. Giuffrida, “SoK: Benchmarking flaws in systems security,” in *Proc. 4th IEEE EuroS&P*. IEEE, 2019, pp. 310–325.
- [S32] T. Hoefler and R. Belli, “Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results,” in *Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC)*. ACM, 2015, pp. 73:1–73:12.
- [S33] J. Li, N. K. Sharma, D. R. K. Ports, and S. D. Gribble, “Tales of the tail: Hardware, OS, and application-level sources of tail latency,” in *Proc. ACM SoCC*. ACM, 2014.
- [S34] P. B. Danzig and S. Melvin, “High resolution timing with low resolution clocks and a microsecond resolution timer for sun workstations,” *ACM SIGOPS Operating Systems Review*, vol. 24, no. 1, pp. 23–26, Jan. 1990.
- [S35] H. Weyl, “Über die gleichverteilung von zahlen mod. eins,” *Mathematische Annalen*, vol. 77, pp. 313–352, 1916.
- [S36] V. T. Sós, “On the distribution mod 1 of the sequence $n\alpha$,” *Annales Universitatis Scientiarum Budapestinensis, Sectio Mathematica*, vol. 1, pp. 127–134, 1958.
- [S37] L. Schuchman, “Dither signals and their effect on quantization noise,” *IEEE Transactions on Communication Technology*, vol. 12, no. 4, pp. 162–165, 1964.

- [S38] R. A. Wannamaker, S. P. Lipshitz, J. Vanderkooy, and J. N. Wright, “A theory of nonsubtractive dither,” *IEEE Transactions on Signal Processing*, vol. 48, no. 2, pp. 499–516, 2000.
- [S39] G. Tene, “How NOT to measure latency,” 2015, talk, Strange Loop / QCon.
- [S40] —, “HdrHistogram: A high dynamic range histogram,” 2015, available at hdrhistogram.org.
- [S41] S. Yang, J. Jeong, B. Scholz, and B. Burgstaller, “Cloudprofiler: TSC-based inter-node profiling and high-throughput data ingestion for cloud streaming workloads,” *arXiv preprint arXiv:2205.09325*, 2022.
- [S42] EEMBC, “CoreMark-PRO run rules,” <https://github.com/eembc/coremark-pro>, 2026, accessed: Aug. 20, 2026.
- [S43] Standard Performance Evaluation Corporation, “SPEC power and performance benchmark methodology, version 2.3,” https://www.spec.org/power/docs/SPEC-Power_and_Performance_Methodology.pdf, 2025, accessed: Aug. 20, 2026.
- [S44] S. Shalunov, B. Teitelbaum, A. Karp, J. W. Boote, and M. J. Zekauskas, “A one-way active measurement protocol (OWAMP),” Internet Engineering Task Force, RFC 4656, Sep. 2006.
- [S45] ETSI, “Speech and multimedia transmission quality (STQ); QoS aspects for popular services in mobile networks; part 4: Requirements for quality of service measurement equipment,” European Telecommunications Standards Institute, Technical Specification TS 102 250-4 V2.3.1, Nov. 2019.
- [S46] G. Almes, S. Kalidindi, M. Zekauskas, and A. Morton, “A one-way delay metric for IPPM,” Internet Engineering Task Force, RFC 7679, Jan. 2016, obsoletes RFC 2679.
- [S47] R. Durairajan, S. K. Mani, P. Barford, R. Nowak, and J. Sommers, “TimeWeaver: Opportunistic one way delay measurement via NTP,” in *Proc. 30th International Teletraffic Congress (ITC 30)*, 2018, pp. 185–193.
- [S48] Standard Performance Evaluation Corporation, “SPECjms2007 run and reporting rules, version 1.0,” <https://www.spec.org/jms2007/docs/RunRules.html>, 2014, accessed: Aug. 20, 2026.
- [S49] K. Sachs, S. Kounev, J. Bacon, and A. P. Buchmann, “Performance evaluation of message-oriented middleware using the SPECjms2007 benchmark,” *Performance Evaluation*, vol. 66, no. 8, pp. 410–434, 2009.
- [S50] A. Klenik and A. Pataricza, “Adding semantics to measurements: Ontology-guided, systematic performance analysis,” *Acta Cybernetica*, 2022, [arXiv:2112.11270](https://arxiv.org/abs/2112.11270).
- [S51] A. Sharma, D. Shah, D. Lariviere, and H. ElBakoury, “Time, causality, and observability failures in distributed AI inference systems,” *arXiv preprint arXiv:2604.21361*, Apr. 2026, Open Compute Project, Unified Intelligent Infrastructure workstream.
- [S52] OpenMessaging Benchmark contributors, “Can the average publish latency be larger than end-to-end latency? (issue #216),” <https://github.com/openmessaging/benchmark/issues/216>, 2021.
- [S53] Jaeger contributors, “model/adjuster/clockskew.go,” <https://github.com/jaegertracing/jaeger>, 2026, version 1.57.0. Accessed: Aug. 21, 2026.
- [S54] —, “Clock skew adjuster considered harmful (issue #1459),” Issue tracker, 2019, <https://github.com/jaegertracing/jaeger/issues/1459>. Accessed: Aug. 21, 2026.
- [S55] OpenTelemetry eBPF Instrumentation contributors, “Child span start time precedes parent span start time (issue #2568),” Issue tracker, 2026, <https://github.com/open-telemetry/opentelemetry-ebpf-instrumentation/issues/2568>. Accessed: Sep. 13, 2026.
- [S56] J.-P. Lozi, B. Lepers, J. Funston, F. Gaud, V. Quéma, and A. Fedorova, “The Linux scheduler: a decade of wasted cores,” in *Proc. 11th European Conf. Computer Systems (EuroSys)*. ACM, 2016, pp. 1–16.
- [S57] A. Chandrasekar and J. Kramberger, “Identifying and mitigating systemic measurement bias in production LLM inference benchmarks,” *arXiv preprint arXiv:2605.24217*, 2026.
- [S58] A. Tahir, B. Alt, A. Rizk, and H. Koepl, “Load balancing in compute clusters with delayed feedback,” *IEEE Transactions on Computers*, vol. 72, no. 6, pp. 1610–1622, 2023.